

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Business Process Automation with Power Automate and Copilot Studio Agents

TGS Ref No: TGS-2022017524

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 3.8

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	24 Jun 2026	Initial release — full 3-day, 17-lab learner guide.	Course Development Team
2.0	2 Jul 2026	WSQ revision — new course title, labs updated to the current Copilot Studio / Power Automate UI, Course Sandbox environment, WSQ cover page.	Course Development Team
3.0	3 Jul 2026	Course restructured from 3 days to 2 days — Day 1: Power Automate (Labs 0-5), Day 2: Copilot Studio agents (Labs 6-11) ending with the WSQ assessment. Modules 4-5 and Labs 12-16 retired.	Course Development Team
3.1	24 Jul 2026	Added Day 1 Labs 6A-6B: external online-form and browser-chatbot webhooks using the Power Automate HTTP Request trigger, with both new and classic designer guidance.	Course Development Team
3.2	24 Jul 2026	Reframed Lab 7 as the complete Copilot Studio IT Support RAG Chatbot outcome: approved FAQ retrieval, citations, negative testing, and grounded refusal.	Course Development Team
3.3	24 Jul 2026	Restructured Labs 8-10 into two-part Teams and website experiences: ordinary HTTP Power Automate flow, deterministic agent flow, and guarded AI prompt flow.	Course Development Team
3.4	24 Jul 2026	Day 2 now concludes at Lab 10. Retired Lab 11 and allocated the remaining guided time to integrated testing, troubleshooting and recap.	Course Development Team
3.5	24 Jul 2026	Made importable	Course Development

		Power Automate packages the recommended classroom path and added a ready-made Lab 8 website enquiry flow package.	Team
3.6	24 Jul 2026	Made natural-language flow creation the primary Lab 1 route: prompt Copilot, inspect and correct the generated draft, then test; retained manual and import recovery routes.	Course Development Team
3.7	24 Jul 2026	Reorganised Copilot Studio Labs 6-10 into two coherent projects: prompt-created IT Support agent upgraded with RAG, then one Marina Trust agent progressively upgraded with HTTP, deterministic and AI prompt flows.	Course Development Team
3.8	24 Jul 2026	Added a visual architecture flowchart to every lab, standardised manual-build and packaged-import routes, and clarified how a Copilot Studio agent calls Power Automate agent flows as tools.	Course Development Team
3.9	24 Jul 2026	Reordered the labs from simple to complex: instant, scheduled, automated, human approval, HTTP, agent creation, RAG, channel deployment, deterministic agent flow and controlled prompt flow.	Course Development Team

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Welcome! This Learner Guide takes you **click-by-click** through every hands-on lab in the WSQ course **Business Process Automation with Power Automate and Copilot Studio Agents** (Course Code: TGS-2022017524). Over two days you go from your first Power Automate flow to AI business agents in Microsoft Copilot Studio — and finish by connecting an agent to your flows in a complete end-to-end automated workflow.

Work through the labs **in order**: each one builds on the skills of the lab before it. Whenever you see a **Checkpoint**, stop and confirm your flow or agent behaves as described before moving on. The **Common Errors & Quick Fixes** and per-lab **Troubleshooting** tables will get you unstuck fast.

Note: Course flow at a glance — Day 1: Workflow automation concepts + Power Automate (Labs 0-5 and webhook Labs 6A-6B). Day 2: Business agents in Copilot Studio + agent-and-flow end-to-end workflows (Labs 6-10), then the WSQ assessment (4:00-6:00 PM).

Common Errors & Quick Fixes

Keep this handy — these are the issues learners hit most often, with the one-line fix:

Symptom	Cause	Fix
"Unauthorized" when sending email	Outlook connection expired or the account has no mailbox	Reconnect the Office 365 Outlook connection with a mailbox-enabled account; both connections must be green ✓
Approval fails: "valid users in the organization"	Approver typed as an external email, not a tenant user	Pick the approver from the people-picker dropdown (a real user in your tenant; yourself is fine for testing)
Date logs as literal text 'utcNow()'	Expression typed into the field as text	Enter it via the fx / Expression editor so it becomes a coloured token
Excel cell shows #####	The column is only too narrow	Auto-fit the column — the value is fine
An unwanted 'For each' wraps your action	You inserted a list/array value into a single-value field	Use single-value fields (Outcome, trigger inputs); delete the For each and re-add a plain action
Agent can't see its flow	Agent and flow are in different environments	Set both Copilot Studio and Power Automate to the same environment (Course Sandbox)

Day 1 — Foundations & Power Automate

Module 1: Introduction to Workflow Automation

Note: Read this before the Day 1 labs. It explains the "why" behind everything you'll build over the next two days. ~20 minutes.

By the end of this reading you will be able to:

- Explain what a business workflow is and which tasks are worth automating
- Tell the difference between an agent that only **talks** and a workflow that actually **does work**
- Use the four building-block terms — **trigger, actions, outputs, steps** — that recur in every single lab

1. What is business workflow automation?

A **business workflow** is a repeatable series of steps that gets work done. You already run dozens of them by hand every week. For example:

Note: *A customer emails an enquiry → someone records it in a spreadsheet → a salesperson is notified → they reply.*

Four small steps, but each one takes attention, and any of them can be forgotten, delayed, or done inconsistently.

Automation means letting software perform those steps for you — consistently, instantly, and the same way every time — instead of doing them by hand. The best candidates for automation share three traits:

Trait	What it looks like	Everyday example
Repetitive	Done the same way many times	Logging enquiries, sending confirmations
Rule-based	Clear "if this, then that" logic	If amount > \$1,000, get approval
Time-consuming	Manual copying, chasing, re-typing	Re-keying form data into a spreadsheet

What you gain: fewer mistakes, faster turnaround, a complete record of what happened, nothing falling through the cracks — and people freed up for higher-value work that actually needs human judgement.

Note: Rule of thumb: if you find yourself doing the same clicking, copying, and emailing over and over, it is probably a workflow waiting to be automated.

In this course you automate workflows with **two** Microsoft tools that work as a pair:

- **Power Automate** — builds the automated steps (the **flows**) that do the work.

- **Copilot Studio** — builds AI **agents** that understand natural-language requests and feed clean, structured data into those flows.

You'll learn Power Automate first (Day 1), then agents and how to combine them with your flows (Day 2).

2. Standalone agents vs integrated workflows

It helps to be clear, from the very start, about the difference between an agent that just **answers** and a workflow that **acts**.

	Standalone agent	**Integrated workflow**
What it does	Chats and answers questions	Performs real actions across systems
Example	"What's your return policy?"	Logs the enquiry, emails the team, files a ticket
Powered by	Copilot Studio alone	Copilot Studio + Power Automate
Outcome	Information	Action + record + notification

A **standalone agent** is genuinely useful, but it only **talks**. Ask it a question, get an answer — the conversation ends and nothing changes in your business systems.

An **integrated workflow** connects that same agent to Power Automate, so the conversation actually **triggers work**: a record gets saved, an email gets sent, an approval gets started. The customer feels heard **and** the back-office task is already done.

Standalone agent: User asks $\rightarrow$ Agent answers (talk only)

Integrated:

```
graph LR; A[User asks] --> B[Agent answers]; B --> C["└─ triggers a flow ─▶ log + email + approval  
                    (talk AND action)"]
```

That bridge between conversation and action is the heart of this course.

3. Workflow logic: the four building blocks

Every workflow you build — whether a simple flow or a full agent-driven process — is made of these four parts. Learn these terms now; you will use them in *every* lab.

Trigger — *what starts the workflow*

The single event that kicks everything off. Examples:

- An **email is received**
- A **file is uploaded** to a folder
- A **form is submitted**
- A **schedule** is reached (e.g. every morning at 9 AM)

- An **agent** calls the flow

Note: Every flow has **exactly one** trigger, and it does nothing until that trigger fires.

Actions — ***what the workflow does***

The steps that run after the trigger, in order. Examples:

- **Send an email**
- **Add a row** to an Excel table
- **Start an approval** and wait for a decision
- **Post a message** to Teams
- **Create or update** a record

Outputs — ***the data the workflow produces or passes along***

The pieces of information that move between steps. Examples:

- The **sender's email address** from a received email
- The **customer name and product** captured by an agent
- The **approval result** ("Approved" / "Rejected")

Outputs from one step become inputs to the next — in Power Automate this is called **dynamic content**, and it's how data flows down the chain.

Steps — ***the ordered sequence as a whole***

Trigger + actions, arranged top-to-bottom, make up the **steps** of the workflow. Steps can also include:

- **Conditions** — branching with if/else logic (e.g. *if amount > \$1,000, route to a manager*)
- **Loops** — repeat an action for each item in a list

Putting it together

TRIGGER	ACTIONS (steps)	OUTPUTS
Email received	→ 1. Read sender + subject	→ sender, subject
	2. Add a row to Excel	→ logged record
	3. Send notification to sales	→ confirmation

Read that left to right: an event happens, the flow performs a series of steps, and data (outputs) is produced and passed along the way. Keep this **Trigger** → **Actions** → **Output** mental model in your head — it is the spine of everything that follows.

4. How the two days fit together

Day	You build	New skill	What it gives you
Day 1	Flows in Power Automate	Triggers, actions, outputs	The "hands" that do work — email, Excel logging, approvals
Day 2	Agents in Copilot Studio , connected to your flows	Structured capture, tools, agent + flow	The "brain & mouth" that talk to people, produce clean data, and hand it to a flow — a complete end-to-end workflow

Day 1 is all about Power Automate. You'll master the trigger → action → output rhythm by building real flows you can run today.

Next: Module 2: Introduction to Power Automate

Module 2: Introduction to Power Automate

Note: Read this after Module 1 and before the Day 1 labs. ~15 minutes.

This module turns the **Trigger** → **Actions** → **Output** idea from Module 1 into the actual tool you'll use all day: **Power Automate**. By the end you'll recognise the flow types, the common triggers and actions, and the one thing that trips up every beginner — connections.

1. What is Power Automate?

Power Automate is Microsoft's automation platform. It lets you build **flows** — automated sequences of steps that run across Microsoft 365 and hundreds of other apps, with little or no code. You design a flow visually in a browser by stacking one **trigger** and one or more **actions** (exactly the building blocks from Module 1).

With Power Automate you can:

- React to events — an email arrives, a file is uploaded, a form is submitted
- Move and transform data between systems — Outlook, Excel, SharePoint, Teams, Dataverse, and more
- Run approvals, send notifications, and schedule recurring jobs
- Be called by a Copilot Studio **agent** as a tool (you'll do this on Day 2)

You build flows at <https://make.powerautomate.com>, inside the environment you set up in **Lab 0**.

Note: **Low-code, not no-thought.** You don't write code, but you do think like a designer: what starts the flow, what it does, and what data moves between steps.

Two ways to create a flow

Power Automate can create a first draft from a natural-language prompt, or you can assemble the cards manually:

Approach	Best use	Maker responsibility
Create your automation with Copilot / Describe it to design it	Quickly turn a clear requirement into a suggested trigger and actions	Inspect every generated card, connection, field and expression
Start from blank	Learn exact configuration or build when Copilot is unavailable	Select and configure every trigger and action

In Lab 1 you start with a prompt, then open the designer and correct the generated draft. This is the working habit used throughout the course:

Describe → Generate → Review → Test → Improve

Types of flow you'll build in this course

There are four flavours of flow. They differ only in *how they start* — once running, they all do the same kind of work.

Flow type	Started by	Example	Lab
Instant	A person clicking Run / a button	Send email and log test data on demand	Labs 1–2
Scheduled	A clock/timetable (Recurrence)	Daily reminder at 9 AM	Lab 3
Automated	An event	New form response, new email, new file	Lab 4
Human approval	A flow pauses for a decision	Approve or reject a request, then branch	Lab 5
HTTP request	An external system posts JSON	Website enquiry or deterministic browser chatbot	Labs 6A–6B
Agent flow	A Copilot Studio agent	Agent logs a request and returns a result	Lab 9

2. Common triggers

Every flow starts with **exactly one trigger** — the event that kicks it off. These are the ones you'll use most:

Trigger	Connector / name	Fires when...	Used in
Email received	Office 365 Outlook — <i>""When a new email arrives (V3)""</i>	Mail lands in a folder	Further practice
File upload	OneDrive / SharePoint — <i>""When a file is created""</i>	A document is dropped into a folder	Further practice
Form submission	Microsoft Forms — <i>""When a new response is submitted""</i>	Someone submits your form	Lab 4
Schedule	<i>""Recurrence""</i>	A timetable you define	Lab 3

		is reached	
Manual	*"Manually trigger a flow"*	You press Run	Labs 1–2 and Lab 5
HTTP request	Request — *"When an HTTP request is received"*	An external website posts JSON	Labs 6A–6B
Agent call	*"When an agent calls the flow"*	A Copilot Studio agent runs the flow as a tool	Labs 9–10

Note: Tip — build with a manual trigger first. When learning a new flow, start with a **manual** trigger so you can press Run and perfect the actions. Once they work, swap in the real trigger (form, email, schedule). The actions stay exactly the same — only the start event changes.

3. Creating workflow actions

After the trigger, you add **actions** — the work the flow performs. These are the core actions you'll lean on throughout the course:

Send emails

Office 365 Outlook → Send an email (V2). Send confirmations, notifications, and digests. Use **dynamic content** (outputs from earlier steps) to personalise the subject and body. *(Lab 1)*

Create Excel entries

Excel Online (Business) → Add a row into a table (and *List rows present in a table*). Log every record into a spreadsheet **table** — this becomes your audit trail and single source of truth. *(Lab 2)*

Note: Excel actions only see data that lives inside a proper **Table** (Insert → Table), not loose cells. This is a classic first-time gotcha.

Notifications & approvals

- *Approvals → Start and wait for an approval.* Pause the flow until a person approves or rejects, then branch on the **Outcome**. *(Lab 5)*
- Notifications to **Teams** or email keep the right people informed at each step.

Actions are wired together with:

- **Dynamic content** — one step's output becomes the next step's input (Module 1's "outputs")
- **Conditions** — if/else branching to route the process

Note: Heads-up on approvals: the approver you pick must be a real **user in your Microsoft 365 tenant**. Approvals can't be sent to an outside personal email address — pick someone in your organisation (yourself is fine for testing).

A note on expressions (fx)

Sometimes a field needs a small calculation or formatting — today's date, trimmed text, a number comparison. Power Automate provides an **expression editor** (the **fx** button) for this. You'll only need a few simple ones; we'll point them out in the labs when they appear.

4. Connections — how Power Automate reaches your apps

Each connector (Outlook, Excel, Approvals, Forms...) needs a **connection** — a saved sign-in that authorises the flow to act on your behalf. The first time you use a connector, you'll **sign in and consent**.

This is the **number one source of "why won't my flow run?"** for beginners, so commit it to memory:

Connection state	What you'll see	What to do
Healthy	Green / connected	Good to go
Broken / expired	Red △ "Reconnect"	Reconnect before running
Wrong rights	"Unauthorized" error	The account lacks rights (e.g. no mailbox) — fix or use a different account

Note: The golden rule for every lab: green connection = ready; red connection = reconnect first. An **Unauthorized** error almost always means: reconnect the connector or check that the signed-in account actually has permission for that action.

5. Importing a packaged Power Automate flow

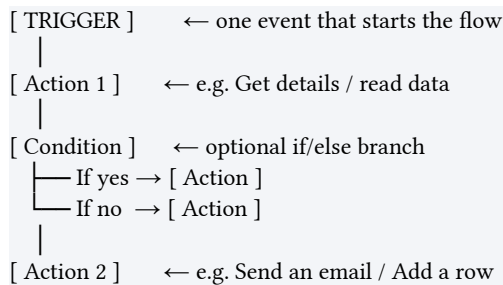
Every flow-based lab includes its own ZIP beside the lab's [index.md](#).

1. Download the individual lab ZIP. Do not extract it.
2. In Power Automate, select the correct **Course Sandbox** environment.
3. Open **My flows** → **Import** → **Import Package (Legacy)**.
4. Upload the ZIP.
5. For the flow, choose **Create as new**.
6. For every connector, choose an existing connection or create one and sign — in.
7. Select **Import**, open the imported flow and complete the lab's — **After import** checklist.
8. Select tenant-owned resources such as the Microsoft Form, Excel workbook, — table, recipient or approver.

9. **Save** → **Test**, then inspect the run history.

Note: **Why connection selection remains:** reusable packages must not contain passwords, access tokens, personal email addresses, tenant IDs, Form IDs or OneDrive file IDs. The flow logic is packaged; Microsoft still requires the learner to authorise resources in their own environment.

6. Anatomy of a flow (what you'll see in the designer)



The build-and-verify loop is the same in every lab:

1. **Save** the flow.
2. **Test** it — manually, or by triggering the real event.
3. Review the **run history** — **green = success, red = error** — to confirm it worked or to debug.

Get comfortable with this loop today; you'll repeat it dozens of times across the two days.

Next: Lab 1: Instant Email Flow

Lab 0: Environment Setup — Create Your Copilot Studio & Power Automate Accounts

Lab Title

Environment Setup — Create Your Microsoft 365, Power Automate, and Copilot Studio Accounts

Lab Objectives

By the end of this lab, you will be able to:

1. Obtain a Microsoft 365 **work or school** account that can use the Power Platform
2. Create a dedicated Power Platform **Sandbox environment** (with Dataverse) for this course

3. Sign in to Power Automate and Copilot Studio and point **both** at the same environment
4. Confirm Outlook and Excel (OneDrive) are available for the later labs
5. Run a verification checklist so you start Lab 1 with zero surprises
6. Understand the difference between a work/school account and a personal account

Prerequisites

- A web browser (Microsoft Edge or Google Chrome recommended)
- A mobile phone (used once for security verification)
- A credit card *(only if you create a new Business trial in Option B — it is **not** charged during the free month)*
- About 30–40 minutes

Note: Tip — Which account should I use? - Option A — You already have a Microsoft 365 work/school account (e.g. [name@company.com](#) provided by your organization). Try this first — you may already have everything you need. Skip Option B and go straight to **Step 1**. - Option B — You do NOT have a work/school account (you only have a personal [@outlook.com](#) / [@gmail.com](#)). Power Automate and Copilot Studio require a *work or school* account, so you'll create one via a free **Microsoft 365 Business trial**. Do **Option B** first, then continue from Step 1.

Note: ⚠ Warning — Why not the Microsoft 365 Developer Program? As of 2024, Microsoft restricted the free Developer Program E5 sandbox to people with an active **Visual Studio Enterprise or Professional subscription**. If you sign up without one you'll see **"You don't currently qualify for a Microsoft 365 Developer Program sandbox subscription."** — so we **do not** use that path in this course. Use **Option B** instead.

Workflow Visual

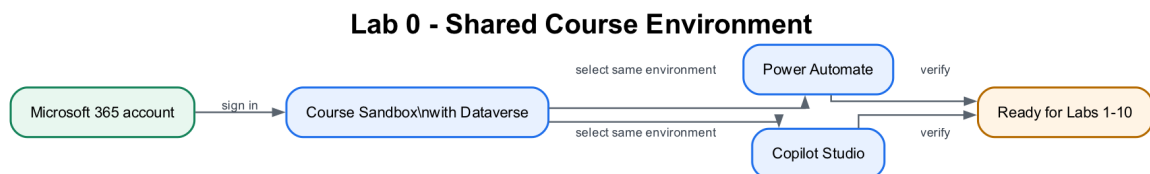


Figure: Lab 0 shared course environment flowchart

The same work or school account and Course Sandbox environment connect every Power Automate flow and Copilot Studio agent used later.

Packaged Flow

No flow package applies to Lab 0 because this lab creates and verifies the environment before any flow exists. The first importable flow is supplied in Lab 1.

Scenario

You have joined **ACME Pte Ltd's Digital Operations project team** as a junior automation specialist. The production tenant contains customer and employee data, so the project manager will not allow experiments there. Your first task is to prepare a controlled **Course Sandbox** where flows, connectors, agent knowledge and test records can be built safely.

Workplace detail	Lab interpretation
Your role	Junior automation specialist
Stakeholders	Power Platform administrator, customer-operations manager and IT security
Operational risk	A learner accidentally sends test emails or writes data into a production system
Success measure	Power Automate and Copilot Studio use the same sandbox environment and all required connections can be verified

Real-world extension: An organisation would also apply environment roles, Data Loss Prevention policies, service accounts, naming standards and a development → test → production deployment process.

Step-by-Step Guide

Option B (only if you have no work/school account): Create a free Microsoft 365 Business trial (~15 minutes)

This creates a brand-new *work* account such as admin@yourname.onmicrosoft.com with Microsoft 365 (Outlook, Excel, OneDrive, SharePoint) — exactly what Power Automate and Copilot Studio need. Skip this entirely if you already have a work/school account.

1. Open a browser and go to <https://www.microsoft.com/microsoft-365/business> (or search "Microsoft 365 Business Standard free trial").
2. Choose **Microsoft 365 Business Standard** and select **Try free for 1 month**.

3. Enter an email address to start. When prompted, choose **Set up account / Create a new account**.
4. Fill in your details (name, business name — you may use your own name, country, phone for verification).
5. Create your **sign-in details**: a username and a domain, giving you something like admin@yourname.onmicrosoft.com. **Write these down** — this is the account you'll use for the entire course.
6. Verify with the code sent to your phone.
7. Add a **payment method** (credit card). You are **not charged during the 1-month free trial**.
8. Wait 1–2 minutes for provisioning. You now have a Microsoft 365 work tenant.

Note: **Tip:** In a classroom, your trainer may provide a ready-made account. Ask before creating a trial so you don't duplicate accounts.

Note: ⚠ **Warning — Avoid surprise charges.** If you don't intend to keep the trial, set a calendar reminder and cancel **before the renewal date** in the **Microsoft 365 admin center** → **Billing** → **Your products**.

Step 1: Sign in to the Microsoft 365 portal (~5 minutes)

1. Go to `<a href="https://www.office.com" target="_blank" rel="noopener">https://www.office.com</a>` (or `<a href="https://m365.cloud.microsoft" target="_blank" rel="noopener">https://m365.cloud.microsoft</a>`).
2. Select **Sign in** and enter your **work/school account** (Option A) or your new **Business trial account** (Option B), then your password.
3. If this is your first sign-in, you may be asked to set up multi-factor authentication (MFA). Follow the prompts using your mobile phone.
4. Once signed in, you should see the Microsoft 365 home page with app tiles (Outlook, Word, Excel, etc.).
5. Open **Outlook** (click the Outlook tile) and send yourself a quick test email to confirm it works — you'll rely on Outlook in Lab 1.
6. Open **Excel** and create a blank workbook to confirm it saves to **OneDrive** — you'll rely on this in Lab 2. You can discard the test workbook afterwards.

Note: ⚠ **Warning — No Outlook/Excel tiles?** Your account may not have a Microsoft 365 license. The **Send an email** action in Lab 1 fails with **"Unauthorized"** when the

account has **no mailbox**. Ask your IT administrator to assign a license, or use the Business trial account from Option B (which always has a mailbox).

Step 2: Create your "Course Sandbox" environment (~7 minutes)

An **environment** is a container that holds your flows, agents, and data. For this course we'll create a dedicated **Sandbox** environment with **Dataverse** turned on, so the Day 2 Copilot Studio agents have a database to use.

1. Open a new tab and go to the **Power Platform admin center**: `<a href="https://admin.powerplatform.microsoft.com" target="_blank" rel="noopener">https://admin.powerplatform.microsoft.com</a>`.
2. Sign in with the **same account** from Step 1.
3. In the left menu, select **Manage** → **Environments**.
4. Select **+ New** (top of the page).
5. Fill in the **New environment** panel: — **Name**: **Course Sandbox** — **Type**: **Sandbox** — **Region**: your nearest region (e.g. Asia, Singapore) — **Add a Dataverse data store**: **Yes** ← important
6. Select **Next**, accept the defaults (language English, currency your local currency) and set **Security group** to **None** (the field is required — *None* means open access), then select **Save**.
7. Wait 1–3 minutes. The environment appears in the list with status **Ready**. Refresh if needed.

Note: Tip: If your tenant blocks creating environments (some organizations restrict this), use the existing **default** environment instead — just remember to select that *same* environment in both Power Automate and Copilot Studio in the next steps.

Step 3: Sign in to Power Automate and select the environment (~5 minutes)

Power Automate is where you build the automated workflows (called **flows**).

1. Open a new tab and go to `<a href="https://make.powerautomate.com" target="_blank" rel="noopener">https://make.powerautomate.com</a>`.
2. Sign in with the **same account**.
3. The first time, you may be asked to **select your country/region** — choose the correct one and select **Get started**.

4. You'll see the Power Automate home page with a left-hand menu: **Home, Create, Templates, Learn, My flows**, plus pinned items such as **Approvals** and a **More** menu (items like **Connections** live under **More**).
5. Look at the **top-right corner** — there is an **Environment selector**. Click it and choose **Course Sandbox** (the one you created in Step 2). All your flows will be built here.
6. Select **My flows** in the left menu — it will be empty for now. That's expected.

Note: ⚠ **Warning:** The environment selector is the single most common source of "where did my flow go?" confusion. If you build a flow in the wrong environment, it simply won't appear when you switch. Always confirm **Course Sandbox** is showing top-right before you build.

Note: **Tip — Free Power Automate:** A Power Automate use-rights plan is included with most Microsoft 365 licenses, which is enough for this course. If prompted, you can also start a **free 90-day trial** of Power Automate Premium.

Step 4: Sign in to Copilot Studio and match the same environment (~5 minutes)

Copilot Studio is where you build the AI **agents** (used on Day 2).

1. Open a new tab and go to `<a href="https://copilotstudio.microsoft.com" target="_blank" rel="noopener">https://copilotstudio.microsoft.com</a>`.
2. Sign in with the **same account** again.
3. If prompted, select your **country/region** and select **Start free trial** (or **Try free**). This activates a **30-day Copilot Studio trial** at no cost (when it expires you can extend it once by another 30 days).
4. Wait for the workspace to load. You'll see the Copilot Studio home page with options to **Create** an agent.
5. Look at the **Environment selector** in the **top-right corner** and choose **Course Sandbox** — the **same** environment you selected in Power Automate.
6. Do **not** create an agent yet — you'll do that in a later lab. For now, just confirm the page loads in the correct environment.

Note: ⚠ **Warning — Both tools MUST use the same environment.** Your agents (Copilot Studio) and your flows (Power Automate) can only call each other when they live in the **same** environment. If Power Automate shows **Course Sandbox** but Copilot Studio shows **Default** (or vice-versa), they cannot connect. Fix it now by clicking the top-right selector in each tool and choosing **Course Sandbox**.

Step 5: Verify your full setup (~5 minutes)

Run this quick checklist. Each item should already be true if the steps above succeeded.

#	Check	Where
1	I can sign in and see app tiles	https://office.com
2	I can open Outlook and send myself an email	Outlook
3	I can open Excel and it saves to OneDrive	Excel / OneDrive
4	My Course Sandbox environment shows status Ready	https://admin.powerplatform.microsoft.com
5	Power Automate home page loads and Course Sandbox is selected	https://make.powerautomate.com
6	Copilot Studio loads, my trial is active, and Course Sandbox is selected	https://copilotstudio.microsoft.com
7	Power Automate and Copilot Studio show the same environment	Top-right selector in both

If all seven are checked, your environment is ready.

Checkpoint

Note: Workplace evidence: Capture the environment selectors in Power Automate and Copilot Studio plus the green connection status. In a real project, these screenshots form part of the deployment-readiness record.

You should now have:

- A working Microsoft 365 **work/school** account (Option A or B)
- A Power Platform **Sandbox** environment named **Course Sandbox** with **Dataverse = Yes**, status **Ready**
- Power Automate open with **Course Sandbox** selected top-right
- Copilot Studio open (trial active) with **Course Sandbox** selected top-right
- Outlook and Excel (OneDrive) confirmed working

Troubleshooting

Problem	Solution
"You can't sign in here with a personal account"	Power Platform needs a *work/school* account. Use Option B to create one via the Business trial.
"You don't currently qualify for a Microsoft 365 Developer Program sandbox subscription"	The Developer Program now requires a Visual Studio subscription. Use Option B (Business trial) instead.
+ New environment button is greyed out / missing	Your tenant restricts environment creation. Ask an admin, or use the Default environment and select it in both tools.
Environment created but stuck on Preparing	Wait 2–3 minutes and refresh the Environments list; provisioning Dataverse takes a moment.
Power Automate says "no environment"	Refresh, re-select your region, then pick Course Sandbox in the top-right selector.
Copilot Studio "Start free trial" button missing	You may already have a license — just proceed. Otherwise sign out and back in.
Different environment shows in each tool	Click the environment selector (top-right) in both tools and choose Course Sandbox .
No Outlook/Excel tiles	Your account lacks a Microsoft 365 license (and possibly a mailbox) — ask IT or use the Business trial account (Option B).

Key Takeaways

- Power Automate and Copilot Studio both need a **work/school** account — personal accounts won't work.
- The **Microsoft 365 Developer Program** is no longer a free path; use a **Business trial** if you need an account.
- An **environment** is the container for your work; this course uses one named **Course Sandbox** (Sandbox type, Dataverse = Yes).
- The **#1 setup mistake** is having the two tools on **different environments** — always verify the top-right selector matches in both.

Duration

~30–40 minutes

Next Steps

Read Module 1: Workflow Automation Concepts and Module 2: Introduction to Power Automate, then proceed to Lab 1: Instant Email Flow.

Lab 1: Create Your First Flow with a Prompt

Lab Title

Create and Verify an Instant Email Flow with Power Automate Copilot

Lab Objectives

By the end of this lab, you will be able to:

1. Describe a business automation in natural language
2. Ask Power Automate Copilot to generate the first version of the flow
3. Review the suggested **trigger**, **action**, connections and field values
4. Correct the flow in the designer instead of assuming AI output is complete
5. Use **dynamic content** to personalise the email
6. **Save, Test** → **Run**, and use the **run history** to verify the result

Prerequisites

- Completed Lab 0 (accounts ready)
- Signed in at <https://make.powerautomate.com> with **Course Sandbox** selected (top-right)
- Outlook working with your account (a mailbox-enabled account)

Workflow Visual



Figure: Lab 1 prompt-created email automation flowchart

The Copilot prompt creates a draft; the learner verifies the trigger, dynamic content and Outlook action before testing.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning how Copilot creates a draft and how to verify it.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder for a faster start or recovery.

Download Lab1-Send-Confirmation-Email.zip, then use **My flows** → **Import** → **Import Package (Legacy)**. Map the Outlook connection and follow the import details.

Use this only when Copilot is unavailable or you need a known-good recovery flow. The main learning route below begins with a natural-language prompt.

Scenario

You are an **ACME Customer Service Officer** handling enquiries received by telephone and at the service counter. During a call, you press **Run**, enter the customer's name and send an acknowledgement immediately. The email sets a one-business-day response expectation and gives the customer confidence that the request was captured.

Workplace detail	Requirement
Trigger	A staff member deliberately runs the flow after verifying the caller's name
Recipient	The controlled training mailbox standing in for the customer
Service target	Acknowledgement sent within 15 minutes of first contact
Evidence	Successful run history plus the personalised email received

Instead of assembling the first version card by card, you describe the requirement to Copilot and then inspect every generated field. This establishes the course pattern:

Describe → Generate → Review → Test → Improve

Part 1 — Build the Flow Step by Step

Step 1: Generate the first draft with Copilot (~7 minutes)

1. Go to <https://make.powerautomate.com> and sign in.
2. Confirm the **Environment selector** (top-right) shows **Course Sandbox** — the environment from Lab 0.
3. On **Home** or **Create**, find **Create your automation with Copilot** or **Describe it to design it**. The wording varies slightly between the new and classic navigation.
4. Paste this prompt, replacing the email placeholder with your own mailbox-enabled course address:

Create an instant cloud flow named Lab 1 - Send Confirmation Email.
Start it with Manually trigger a flow and ask for a text input named CustomerName. Then use Office 365 Outlook Send an email (V2) to YOUR_EMAIL@YOUR_TENANT. Use the subject 'Thank you for your enquiry'.
The body should greet the CustomerName and say that ACME Customer Operations has logged the enquiry and will respond within 1 business day.

1. Submit the prompt. Read the proposed structure; it should contain **Manually trigger a flow** → **Send an email (V2)**.
2. If the proposed trigger or action is wrong, add this refinement:

Use only Manually trigger a flow and Office 365 Outlook Send an email (V2). Do not use Gmail, Outlook.com, Teams, or an automated trigger.

1. Select **Keep it and continue**. Check that the Outlook connection has a green check, sign in if required, then select **Create flow**.

Note: Important: Copilot creates a draft, not proof that the automation is correct. The next step is compulsory: inspect every generated card and value.

Note: Can't see the Copilot prompt box? Your tenant, region or licence may not expose natural-language flow creation. Use **Create** → **Instant cloud flow**, name it **Lab 1 - Send Confirmation Email**, select **Manually trigger a flow**, and continue below. Alternatively use the recovery import package.

Step 2: Review and correct the generated flow (~8 minutes)

1. Rename the flow to **Lab 1 - Send Confirmation Email** if Copilot used a different name.
2. Confirm the first card is **Manually trigger a flow**.
3. Open the trigger. Confirm it contains one **Text** input named **CustomerName**.
4. If the input is missing, select **+ Add an input** → **Text**, then name it **CustomerName**.
5. Confirm the next card is **Send an email (V2)** from **Office 365 Outlook**. Delete and replace it if Copilot selected Gmail, Outlook.com or another connector.

Note: Tip: This input becomes one of the trigger's **outputs** — a piece of data you can drop into later steps using dynamic content.

Step 3: Verify and complete the email action (~10 minutes)

If Copilot did not add an email action, select the **+** below the trigger, choose **Add an action**, search for **Send an email**, and select **Office 365 Outlook** → **Send an email (V2)**.

Note: ⚠ **Warning:** Pick **Office 365 Outlook**, not Gmail, Outlook.com, or SMTP. Only Office 365 Outlook uses your course work account.

1. If required, select **Sign in**, choose your course account, and approve the Outlook connection. A green ✓ means it is ready.
2. Open the generated email action and verify every field. Do not keep a value merely because Copilot supplied it.

3. Configure the email fields using the copy-paste blocks below.

- **To:** type your own email address, then press **Enter** so it resolves into a **chip** (a small pill with an × next to it). If it stays as plain text, the address wasn't accepted — retype it.
- **Subject:** copy-paste this line:

Thank you for your enquiry

- **Body:** build the message in three parts — paste, insert token, paste:

1. Click inside the **Body** field, paste this text, and then type one **space**:

Hi

1. Select the **dynamic content** icon (the small lightning bolt) that appears in/next to the field.
2. From the list, under **Manually trigger a flow**, choose **CustomerName**. It appears as a coloured **token** (chip) in the field.
3. Click just after the token and paste the rest:

, thank you for contacting ACME Customer Operations. Your enquiry has been logged and a service officer will respond within 1 business day.

Note: **Tip: Dynamic content** is how outputs from earlier steps get reused. The coloured **CustomerName** token is a placeholder — it's replaced with the real value when the flow runs.

Note: ⚠ **Warning — paste as plain text.** If you copy from a PDF or Word version of this guide, formatting (smart quotes “ ”, curly apostrophes, hidden line breaks) can come along and appear literally in the email. Paste with **Ctrl+Shift+V** (Mac: **Cmd+Shift+V**) to strip formatting, or copy from the Markdown code boxes above, which contain plain text only. Never copy backticks (` ``) into a field.

Note: **Check before saving:** To shows a **chip**, Subject is plain text, Body reads **Hi [CustomerName-token], thank you for reaching out...** with exactly one coloured token. If **CustomerName** appears as plain black text instead of a token, delete it and re-insert from the dynamic content list.

Step 4: Save and test (~5 minutes)

1. Select **Save** (top-right).

Note: **Tip:** There is **no separate "Send" button**. Running the flow **is** what sends the email — the **Send an email** action does the work. So the routine is always: **Save**, then **Test** → **Run flow**.

1. Select **Test** (top-right) → choose **Manually** → **Test** → **Run flow**.

2. Power Automate prompts for the input you defined: — **CustomerName:** type **Jane Tan**
3. Select **Run flow**, then **Done**.
4. Watch the run status — each step should show a **green check**.
5. Open **Outlook** and confirm the email arrived, personalized as "Hi Jane Tan".

Step 5: Review the run history (~5 minutes)

1. In the left menu, select **My flows**, then open **Lab 1 - Send Confirmation Email**.
2. Look at the **28-day run history** — you'll see your test run with a status.
3. Select the run to inspect each step's **inputs and outputs**. This is how you debug flows: a **green check** = success; a **red**  = error (click it to read the message).

Part 2 — Import the Packaged Flow

If you get stuck, use either of these packages:

- **Legacy flow package:** Lab1-Send-Confirmation-Email.zip for **My flows** → **Import** → **Import Package (Legacy)**.
- **Dataverse solution:** Lab1-Send-Confirmation-Email-Solution.zip for **Solutions** → **Import solution**.

For the solution route:

1. Confirm the **Environment selector** (top-right) shows **NUS Copilot Sandbox**.
2. In the left menu, select **Solutions** → **Import solution** (toolbar).
3. **Browse** → choose the ZIP → **Next**.
4. On the **Connections** page, the **Office 365 Outlook** connection reference asks for a connection — pick an existing one or **+ New connection** (sign in with your course account), then **Import**.
5. When the import completes, open the solution **Lab 1 - Send Confirmation Email** → open the flow → **Edit**, change the **To** address to your own email, and **Save**. Turn the flow **On** if it shows as Off.
6. Continue from Step 4: Save and test.

Note: **Tip:** Importing gives you a known-good flow definition — if the imported flow **still** fails, the problem is your **connection/account** (see Troubleshooting), not the flow.

Note: After a legacy import, open the flow, reconnect Outlook, replace **YOUR_EMAIL@YOUR_TENANT** with your email, then save and test. If your tenant says flows must be created in Dataverse solutions, use the solution package.

Checkpoint

Note: Workplace evidence: Retain the successful run-history screen and the received acknowledgement email. A supervisor should be able to match the input customer name to the message that was delivered.

You should now have:

- A flow named **Lab 1 - Send Confirmation Email** with a **CustomerName** text input
- ✓ An Office 365 Outlook connection showing a green checkmark
- A successful test run (all steps green)
- A personalized email received in Outlook reading "Hi Jane Tan, ..."

Troubleshooting

Problem	Solution
No "Send an email (V2)" action listed	Make sure you selected the Office 365 Outlook connector (not Gmail / Outlook.com / SMTP).
Action fails with " Unauthorized "	The Office 365 Outlook connection is broken/expired, or the signed-in account has no mailbox . Open the action's connection, reconnect with a mailbox-enabled account (green ✓).
BadRequest — "content was not a valid JSON ... parsing value: R"	The connection account has no Exchange Online mailbox (Exchange replies with plain text like "REST API is not yet supported for this mailbox", which starts with "R"). Verify by signing in at outlook.office.com with that account. Fix: use a connection with a mailbox-enabled account , or ask the admin to assign an Exchange Online license. Re-editing the flow will not fix this.
Connection shows a red ⚠	The connection needs to be re-authorized — select it and reconnect / sign in again.
Email not received	Check Junk/Spam; confirm the To address; re-run the test.
Can't find dynamic content	Click directly inside the Body field first, then open the lightning-bolt menu.
CustomerName shows as plain text, not a token	Delete it and re-insert it from the dynamic content list so it becomes a coloured token.
Clicked "Save" but no email arrived	Saving does not send. You must Test → Run flow — running the flow performs the action.

Key Takeaways

- A clear prompt states the trigger, connector, action, inputs and expected output.
- Copilot accelerates the first draft; the maker remains responsible for reviewing connections, fields, dynamic content and behaviour.

- Every flow follows the pattern **Trigger** → **Actions**.
- Trigger **inputs** become **outputs** that you reuse in later steps via **dynamic content** tokens.
- A connector needs a **connection**: green ✓ = ready, red ⚠ = reconnect.
"Unauthorized" on *Send an email* means the Outlook connection is broken or the account has no mailbox.
- There is **no separate Send button** — **Save**, then **Test** → **Run** to make the actions happen.
- **Test + run history** are your tools for verifying and debugging.

Duration

~30 minutes

Next Steps

Proceed to Lab 2: Instant Excel Logging Flow.

Lab 2: Instant Excel Logging Flow

Lab Title

Capture Form Data and Log It into Excel with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

1. Prepare an Excel workbook with a formatted **Table** in OneDrive
2. Build a flow with a **manual trigger** that collects Name, Email, and Message
3. Add an **Add a row into a table** (Excel) action and connect to your workbook
4. Map trigger outputs into Excel columns and stamp the date using an **fx expression**
5. **Save**, then **Test** → **Run** and verify each submission appears as a new row
6. Understand how to swap the manual trigger for a real Microsoft Forms trigger

Prerequisites

- Completed Lab 1
- Access to Excel via OneDrive (verified in Lab 0)
- Signed in at <https://make.powerautomate.com> with **Course Sandbox** selected (top-right)

Workflow Visual

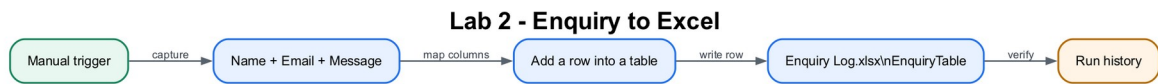


Figure: Lab 2 enquiry-to-Excel flowchart

Each trigger input maps to one named column in the Excel table.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning Excel table mapping.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder, then

select your own workbook and table.

Download Lab2-Log-Enquiry-to-Excel.zip, then

use **My flows** → **Import** → **Import Package (Legacy)**. Map the Excel connection and follow the import details.

You only need to select your own workbook and table after import.

Scenario

You are the **ACME Customer Operations Coordinator**. Phone calls and walk-in enquiries currently live in personal notebooks, so supervisors cannot see workload or prove that a customer was followed up. You will create a shared enquiry register and an instant flow that writes one traceable row for every contact.

Workplace detail	Requirement
Users	Customer-service officers
System of record for this pilot	OneDrive Excel table named EnquiryTable
Minimum audit data	Timestamp, customer name, contact email, request and status
Success measure	Every test run adds exactly one complete row with status New

Production extension: Replace Excel with Dataverse or a CRM when concurrent users, permissions, retention rules and reporting become important.

Part 1 — Build the Flow Step by Step

Step 1: Create the Enquiry Log workbook with a Table (~10 minutes)

Power Automate can only read and write Excel data that is formatted as a **Table** — not loose cells.

1. Go to `<a href="https://office.com" target="_blank" rel="noopener">https://office.com</a>`, open **Excel**, and create a **New blank workbook**.
2. Rename it (click the file name at the top of the screen): **Enquiry Log**. It saves automatically to **OneDrive**.
3. In **row 1**, type these five column headers, one per cell (headers must be in row 1): — A1: **Date** — B1: **Name** — C1: **Email** — D1: **Message** — E1: **Status**
4. Select the header range **A1:E1**.
5. On the ribbon, select **Insert** → **Table**.
6. In the **Create Table** dialog, tick **My table has headers**, then select **OK**.
7. With the table selected, open the **Table Design** tab and set **Table Name** to **EnquiryTable**. This makes it easy to find in Power Automate.
8. Close the Excel tab — changes are saved to OneDrive automatically.

Note: ⚠ **Warning:** If you skip **Insert** → **Table**, your file will have data but **no Table**, and Power Automate's "Table" dropdown will be empty in Step 3. The Table is mandatory.

Note: **Tip:** Naming the table **EnquiryTable** is optional but strongly recommended — otherwise it appears as **Table1** and is harder to pick out later.

Step 2: Create the flow and trigger (~5 minutes)

We'll use a manual trigger with inputs to *simulate* a submitted form. (Microsoft Forms uses the same downstream pattern — see Step 5.)

1. Go to `<a href="https://make.powerautomate.com" target="_blank" rel="noopener">https://make.powerautomate.com</a>`, confirm **Course Sandbox** is selected top-right, then select **Create** → **Instant cloud flow**.
2. **Flow name:** **Lab 2 - Log Enquiry to Excel**
3. **Choose how to trigger this flow:** select **Manually trigger a flow**, then **Create**.
4. Select the trigger card to open its panel, then select **+ Add an input** three times, choosing **Text** each time, and name them: — **Name** — **Email** — **Message**

Step 3: Add the "Add a row into a table" action (~10 minutes)

1. Below the trigger, select the + (plus) button, then **Add an action**.
2. In the search box, type **Add a row into a table**.
3. Select the **Excel Online (Business)** connector → action **Add a row into a table**.
4. If prompted, **Sign in** to create the connection (green ✓ = ready).
5. Configure where your file lives: — **Location:** **OneDrive for Business** — **Document Library:** **OneDrive** — **File:** browse to and select **Enquiry Log.xlsx** — **Table:** select **EnquiryTable**
6. The action now shows one field per column. Fill them in: — **Date:** this must be an **expression**, not typed text. Click the **Date** field, then select the **fx** (Expression) editor. Type exactly:

```
formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')
```

Then select **Add / OK**. The value should appear in the field as a single **coloured token** (chip).

- **Name:** click the field → dynamic content → choose **Name**
- **Email:** click the field → dynamic content → choose **Email**
- **Message:** click the field → dynamic content → choose **Message**
- **Status:** type the static text **New**

Note: ⚠ **Warning — fx, not typing.** The `formatDateTime(...)` expression **must** be entered through the **fx / Expression** editor so it becomes a coloured token. If you just type it into the Date field, Power Automate logs the literal text `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` into the cell instead of the actual date.

Note: **Tip — UTC vs local time.** `utcNow()` returns the time in **UTC**, so a 4pm Singapore enquiry logs as 08:00. If you want Singapore local time, use this expression in the **fx** editor instead: `convertTimeZone(utcNow(),UTC,'Singapore Standard Time','yyyy-MM-dd HH:mm')`

Step 4: Save and test (~10 minutes)

1. Select **Save** (top-right).

Note: **Tip:** As in Lab 1, there's **no separate "submit" button** — running the flow **is** what writes the row. Always **Save**, then **Test** → **Run flow**.

1. Select **Test** → **Manually** → **Test** → **Run flow**.
2. Enter sample values when prompted: — **Name:** **Ahmad Rahman** — **Email:** **ahmad@example.com** — **Message:** **I need the document checklist for opening an SME current account.**

3. Select **Run flow** → **Done**. Confirm every step shows a **green check**.
4. Open **Enquiry Log.xlsx** in Excel — a **new row** should appear with your values, a **New** status, and a timestamp in the Date column.
5. Run the test **2–3 more times** with different values to confirm each submission adds another row.

Step 5: (Optional) Connect it to a real Microsoft Form (~5 minutes)

To turn this into a true *form submission* workflow:

1. Create a form at `<a href="https://forms.office.com" target="_blank" rel="noopener">https://forms.office.com</a>` with three questions: Name, Email, Message.
2. Build a **new** flow with the trigger **When a new response is submitted** (Microsoft Forms connector).
3. Add the **Get response details** action, then the same **Add a row into a table** action, mapping the Forms answers into the columns (and keep the same `formatDateTime(...)` fx expression for Date).

Note: Tip: This shows the power of triggers — swap the manual trigger for a Forms trigger and the *same* logging logic runs automatically on every real submission.

Part 2 — Import the Packaged Flow

Download Lab2-Log-Enquiry-to-Excel.zip, then use

My flows → **Import** → **Import Package (Legacy)**. After import, open the Excel action and reselect your own **Enquiry Log.xlsx** file and **EnquiryTable**; these tenant-owned resources cannot be stored in a reusable package.

Checkpoint

Note: Workplace evidence: Show one successful flow run beside the matching Excel row, including timestamp, customer details and **New** status. This demonstrates traceability from request to register.

You should now have:

- **Enquiry Log.xlsx** in OneDrive with a Table named **EnquiryTable** (headers Date, Name, Email, Message, Status in row 1)

- A flow named **Lab 2 - Log Enquiry to Excel** with Name/Email/Message text inputs
- A **Date** column populated by the `formatDateTime(...)` **fx** expression (a coloured token, not literal text)
- Several test rows logged, each with a timestamp and Status **New**

Troubleshooting

Problem	Solution
File or Table not listed in the action	Ensure the data is formatted as a Table (Insert → Table), saved in OneDrive , with headers in row 1.
Date cell shows the literal text <code>formatDateTime(utcNow(),...)</code>	You typed the expression instead of using fx . Delete it, click the fx / Expression editor, paste the expression, and confirm it becomes a coloured token.
Date cell shows `#####`	This is not an error — the column is just too narrow. Auto-fit the column (double-click its right border) to see the value.
Times look wrong / off by hours	<code>utcNow()</code> is UTC . Use the <code>convertTimeZone(..., 'Singapore Standard Time', ...)</code> expression for local time.
Row added but Name/Email/Message blank	Re-map each column to the correct dynamic content token.
Stray backtick or quote appears in a cell	Don't paste characters from this guide. Type values yourself or use the fx editor.
Connection shows red ⚠	Re-authorize the Excel Online (Business) connection (sign in again) until it shows a green ✓.
Saved but no row appeared	Saving doesn't write rows. You must Test → Run flow — running performs the action.

Key Takeaways

- Power Automate reads/writes Excel **Tables** (Insert → Table), never loose cells.
- The **Date** must be entered via the **fx** editor so `formatDateTime(...)` becomes a token — typing it logs the literal text.
- `utcNow()` is **UTC**; use `convertTimeZone(...)` for Singapore local time.
- `#####` in a cell means the column is too narrow — auto-fit it; it's not an error.
- The same logging action works behind **any** trigger — manual, Microsoft Forms, email, or a Copilot Studio agent.

Duration

~35 minutes

Next Steps

Proceed to Lab 3: Scheduled Flow.

Lab 3: Scheduled Flow

Lab Title

Build a Scheduled (Recurring) Workflow with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

1. Create a **Scheduled cloud flow** that uses the **Recurrence** trigger
2. Configure the Interval, Frequency, **Time zone**, hours, and specific days
3. Send a **daily reminder email** automatically on a timetable
4. **Test/Run now** without waiting for the scheduled time
5. (Optional stretch) Read the Excel **EnquiryTable** and include a live **count** in the email

Prerequisites

- Completed Lab 1 (Send an email)
- *(For the optional stretch)* An Excel **Enquiry Log** workbook with table **EnquiryTable** from Lab 2
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment

Workflow Visual



Figure: Lab 3 scheduled reminder flowchart

The recurrence trigger starts the automation without a person submitting data.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning recurrence rules.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder, bind

Outlook, set the recipient and verify the schedule.

Download Lab3-Scheduled-Enquiry-Reminder.zip,

then use **My flows** → **Import** → **Import Package (Legacy)**. Map the Outlook connection and follow the import details.

The recurrence and reminder action are already configured.

Scenario

You are the **ACME Customer Operations Team Lead**. New enquiries must receive a substantive reply within one business day, but records marked **New** are being missed during busy periods. At **9:00 AM every weekday**, the team needs an operational reminder to review the shared queue before customer calls begin.

Workplace detail	Requirement
Process owner	Customer Operations Team Lead
Trigger	Weekdays at 9:00 AM Singapore time
Control	No weekend notification and no UTC timing error
Success measure	Reminder arrives with the queue review instruction; optional stretch includes the current row count

In production, the reminder would be sent to a shared mailbox or Teams channel and would count only rows whose **Status** is **New**, not every row.

Note: Tip: Trigger types so far — **instant/manual** (Labs 1–2, you press Run) and **scheduled** (this lab, runs by the clock). Lab 4 adds an **automated event** trigger (a form submission).

Part 1 — Build the Flow Step by Step

Step 1: Create a scheduled flow (~6 minutes)

1. Go to <https://make.powerautomate.com>.
2. Top-right, confirm the environment selector reads **Course Sandbox**. If not, click it and switch.
3. In the left menu, click **+ Create**.
4. Under "Start from blank", click **Scheduled cloud flow**.
5. In the dialog: — **Flow name:** **Lab 3 - Scheduled Enquiry Reminder** — **Starting:** today's date and any time — **Repeat every:** **1** and **Week**
6. Click **Create**. The designer opens with a **Recurrence** trigger already added.

Step 2: Fine-tune the schedule (~10 minutes)

1. Click the **Recurrence** trigger card to open its configuration panel.
2. Set the basics: — **Interval:** **1** — **Frequency:** **Week**

Note: Tip: Frequency must be **Week** (not Day) — the **On these days** option in the next step only appears with a weekly frequency.

1. Under **Advanced parameters**, select and set: — **Time Zone:** (UTC+08:00) **Kuala Lumpur, Singapore** — **On These Days:** tick **Monday, Tuesday, Wednesday, Thursday, Friday** (leave Saturday and Sunday unticked) — **At These Hours:** **9** — **At These Minutes:** **0**
2. The schedule now reads: **every weekday at 9:00 AM Singapore time.**

Note: ⚠ **Warning:** You **MUST** set the **Time zone**. Without it, the Recurrence trigger uses **UTC**, so a "9" would fire at 9:00 UTC = 5:00 PM Singapore — the wrong local time. This is the same UTC-vs-local trap from Lab 2.

Step 3: Send the reminder email (~8 minutes)

1. Below the Recurrence trigger, click **+** → **Add an action.**
2. Search **send an email** and select **Send an email (V2)** (from **Office 365 Outlook**). Complete the connection if prompted (it must show a green check).
3. Configure: — **To:** your team's address (use your own mailbox for testing) — **Subject:** **Daily reminder: review new enquiries** — **Body:**

Good morning,
This is your daily reminder to review and follow up on new enquiries in the Enquiry Log. Please action any items marked "New".

1. Top-right, click **Save.**

Note: ⚠ **Warning:** If you hit an **Unauthorized** error, the Outlook connection is broken or the account has no mailbox. Reconnect **Office 365 Outlook** with a mailbox-enabled account (see Lab 1). The connection must show a green ✓ before running.

Step 4: Test the flow now (~5 minutes)

You don't have to wait until 9 AM — you can run it on demand to check it works.

1. Top-right, click **Test** → **Manually** → **Test** → **Run flow** → **Done.**
2. Confirm every step shows a green check and the reminder email arrives in your inbox.
3. From now on the flow also runs **automatically** on its schedule.

Note: Tip: A scheduled flow only fires on its timetable in real life — **Test** → **Run flow** lets you verify it immediately instead of waiting for 9 AM tomorrow.

Step 5 (Optional stretch): Include a live count from Excel (~15 minutes)

Make the reminder smarter by counting how many enquiries are logged in **EnquiryTable**.

1. **Above** the Send an email action, click + → **Add an action**.
2. Search **list rows** and select **List rows present in a table** (from **Excel Online (Business)**).
3. Configure the location: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to **Enquiry Log** (the **.xlsx** workbook) — **Table:** **EnquiryTable**
4. Open the **Send an email (V2)** action again. Click into the **Body** where you want the count, then open the **fx** (expression) editor and enter exactly:

```
length(outputs('List_rows_present_in_a_table')['body/value'])
```

- Click **Add / OK** so it becomes a **token** (highlighted chip), not plain text.
 - Example line: **There are currently** *(token)* **enquiries logged.**
1. Click **Save**, then **Test** → **Run flow** again. The email now shows the live record count.

Note: ⚠ **Warning:** The name inside **outputs('...')** must match your action's **internal name** exactly, with spaces replaced by underscores. If the expression errors, open the **List rows present in a table** action → ... menu → check the action name, and adjust **List_rows_present_in_a_table** to match.

Part 2 — Import the Packaged Flow

Download Lab3-Scheduled-Enquiry-Reminder.zip,

then use **My flows** → **Import** → **Import Package (Legacy)**. Reconnect Outlook, replace **YOUR_EMAIL@YOUR_TENANT**, and confirm the recurrence uses ****Singapore Standard Time**** before saving.

Checkpoint

Note: Workplace evidence: Capture the weekday 9:00 AM recurrence settings and the resulting team reminder. The evidence should make the operating schedule and intended recipient unambiguous.

- A **scheduled** flow **Lab 3 - Scheduled Enquiry Reminder** using the **Recurrence** trigger
- Configured for **weekdays at 9:00 AM** with **Time zone (UTC+08:00) Kuala Lumpur, Singapore**

- A reminder email sent on a successful **Test** → **Run flow**
- *(Optional)* A live record count pulled from **EnquiryTable**

Troubleshooting

Problem	Solution
Email arrives at the wrong time	Set Time Zone in the Recurrence trigger's Advanced parameters (e.g. (UTC+08:00) Kuala Lumpur, Singapore). Without it the schedule uses UTC.
Flow runs every day including weekends	In On These Days , tick only Monday–Friday .
On These Days option is missing	Set Frequency to Week — the days-of-week option only appears with a weekly frequency.
Send email Unauthorized	Reconnect Office 365 Outlook with a mailbox-enabled account; the connection must show green ✓.
length(...) expression error	Match the name inside outputs(...) to the List rows action's actual internal name (spaces become underscores).
Don't want to wait for the schedule	Use Test → Manually → Run flow to run it immediately.
Count shows as literal text, not a number	The expression wasn't added via the fx editor as a token. Re-enter it through fx and confirm it becomes a highlighted chip.

Key Takeaways

- The **Recurrence** trigger runs flows automatically on a timetable — no human starts them.
- Always set the **Time zone** so schedules fire at the right local time, not UTC.
- Use **Test** → **Run flow** to verify a scheduled flow without waiting for its scheduled time.
- Scheduled flows are ideal for digests, reminders, and clean-up jobs.

Duration

~30 minutes (45 with the optional stretch)

Next Steps

Proceed to Lab 4: Automated Form Flow.

Lab 4: Automated Form Flow

Lab Title

Capture Microsoft Forms Submissions to Email and Excel (Automatic Trigger)

Lab Objectives

By the end of this lab, you will be able to:

1. Create a **Microsoft Form** with three required questions and a shareable **URL**
2. Build a flow using the **When a new response is submitted** trigger (Microsoft Forms)
3. Add **Get response details** to read each answer (mapping the Response Id)
4. **Email** the submitted enquiry to your team
5. **Log** the same enquiry into the Excel **EnquiryTable** — all automatically on submit

Prerequisites

- Completed Lab 1 (Send an email) and Lab 2 (Excel table + logging)
- An Excel **Enquiry Log** workbook with table **EnquiryTable** (Date, Name, Email, Message, Status) from Lab 2
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment

Workflow Visual

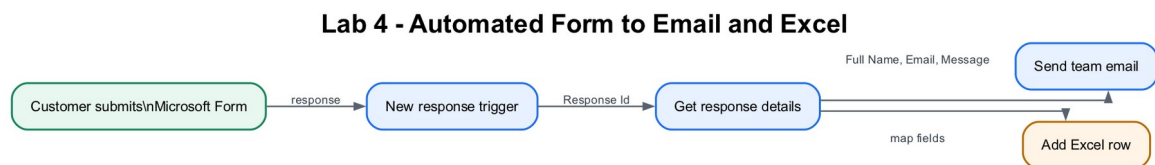


Figure: Lab 4 automated Microsoft Forms workflow flowchart

The Response Id links the Forms trigger to the full response details used by email and Excel.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning Forms dynamic content and Excel mapping.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder, then select your own form, workbook and table.

Download

Lab4-Automated-Form-to-Email-and-Excel.zip,

then use **My flows** → **Import** → **Import Package (Legacy)**. Map Forms, Outlook and Excel connections and follow the import details. Select your

own form, workbook and table after import.

Scenario

ACME is replacing email-based enquiries with a controlled online intake form.

You are the **Customer Operations Automation Specialist** responsible for ensuring that every submission reaches the service mailbox and the shared register without re-keying.

Workplace detail	Requirement
Customer journey	Submit name, email and a detailed service request
Back-office outcome	Notify the team and add a timestamped row with status New
Service target	New submission visible to operations within one minute
Acceptance test	Two different form submissions create two emails and two separate Excel rows

This is your first **automated, event-driven** workflow: Labs 1–2 were instant/manual and Lab 3 was scheduled.

Part 1 — Build the Flow Step by Step

Step 1: Create the form in Microsoft Forms (~10 minutes)

1. Open a new browser tab and go to `<a href="https://forms.office.com" target="_blank" rel="noopener">https://forms.office.com</a>` (it redirects to the current home, `<a href="https://forms.cloud.microsoft" target="_blank" rel="noopener">https://forms.cloud.microsoft</a>` — both work).
2. Sign in with the **same account** you use for Power Automate in the **Course Sandbox** tenant.
3. Click **+ New Form**.
4. Click the title at the top and enter: **Customer Enquiry Form**.
5. Add a short description, e.g. **Tell us about your enquiry and we'll respond within 1 business day**.
6. Add these three questions. For each, click **+ Add new**, choose **Text**, type the question, then toggle **Required** on: — **Full Name** → **Required** on — **Email** → **Required** on — **Your Message** → click the ... on the question and choose **Long answer** for more space → **Required** on
7. The form saves automatically as you type.

Note: **Tip:** Keep the question titles exactly as shown — you'll match them to Excel columns later, and clear names make the dynamic tokens easy to find.

Step 2: Get the shareable form URL (~5 minutes)

1. Top-right of Forms, click **Collect responses** (older UI: **Share**).
2. Set the audience — for testing choose **Anyone can respond** (or your organization).
3. Copy the **link** shown (e.g. <https://forms.office.com/r/XXXXXXXX>).
4. **Save this URL** somewhere — it's the link you'd send to customers, and you'll use it to test at the end.

Step 3: Create the automated flow (~5 minutes)

1. Go back to `<a href="https://make.powerautomate.com" target="_blank" rel="noopener">https://make.powerautomate.com</a>` (confirm the environment is **Course Sandbox**).
2. Left menu → **+ Create** → **Automated cloud flow**.
3. **Flow name:** **Lab 4 - Automated Form to Email and Excel**.
4. In "Choose your flow's trigger", search **Forms** and select **When a new response is submitted** (Microsoft Forms).
5. Click **Create**.
6. On the trigger card, set **Form Id** → pick **Customer Enquiry Form** from the dropdown.

Step 4: Get the response details (~10 minutes)

The trigger only gives you a response **Id** — you need another action to read the actual answers.

1. Below the trigger, click **+** → **Add an action**.
2. Search **Forms** → select **Get response details** (Microsoft Forms).
3. Configure: — **Form Id:** pick **Customer Enquiry Form** (the same form) — **Response Id:** click the field, open dynamic content (lightning bolt), and insert **Response Id** (from the trigger)

Note: ⚠ **Warning: Get response details is required.** Without it, later steps only see an internal Id, not the real answers — and your email/Excel will be blank. Always map **Response Id** from the trigger, then use **this action's** outputs (Full Name, Email, Your Message) in every step that follows.

Step 5: Email the submission to your team (~10 minutes)

1. Click + → **Add an action**.
2. Select **Send an email (V2)** (Office 365 Outlook). Complete the connection if prompted (it must show a green check).
3. Configure (insert each value from **Get response details** dynamic content): — **To:** your team's address (use your own working mailbox for testing) — **Subject:** type **New enquiry from** then insert **Full Name** — **Body:**

A new enquiry was submitted via the form:
Name: [insert Full Name]
Email: [insert Email]
Message: [insert Your Message]

Note: ⚠ **Warning:** If you get an **Unauthorized** error, the Outlook connection is broken or the account has no mailbox. Reconnect **Office 365 Outlook** with a mailbox-enabled account (see Lab 1); the connection must show a green ✓.

Step 6: Log the submission to Excel (~10 minutes)

1. Click + → **Add an action**.
2. Select **Add a row into a table** (Excel Online (Business)). Complete the connection if prompted.
3. Set the location: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to **Enquiry Log** (the **.xlsx** workbook) — **Table:** **EnquiryTable**
4. Map the columns: — **Date:** click the **fx** (expression) icon → enter exactly **formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')** → click **Add / OK** so it becomes a **token** (a highlighted chip), never typed text — **Name:** dynamic content → **Full Name** — **Email:** dynamic content → **Email** — **Message:** dynamic content → **Your Message** — **Status:** type **New**
5. Top-right, click **Save**.

Note: ⚠ **Warning:** The **Date** value must be entered through the **fx** editor and become a token. If you type the formula as plain text, the cell stores the literal text **formatDateTime(...)** instead of a real date. (This is the same fix from Lab 2.)

Step 7: Test the whole workflow (~10 minutes)

1. In Power Automate, click **Test** → **Manually** → **Test**. The flow goes into a "waiting for trigger" state.
2. Open the **form URL** you saved in Step 2 (new tab or your phone).

3. Fill in the form — Full Name **Jane Tan**, Email **jane@example.com**, Your Message **Please send me the document checklist for an SME current account.** — and click **Submit**.
4. Within about a minute the flow triggers. Confirm: — Every step shows a green check in the run. — The **email** arrives with the submitted details. — A **new row** appears in **Enquiry Log** → **EnquiryTable** with a clean date, the answers, and **Status New**.
5. Submit the form a **second time** with different details and confirm another row and another email.

Note: Tip: Triggers can take up to a minute. If nothing happens, confirm you clicked **Test** *before* submitting — or just submit again, since a saved flow fires automatically on every real submission.

Part 2 — Import the Packaged Flow

Download

Lab4-Automated-Form-to-Email-and-Excel.zip,

then use **My flows** → **Import** → **Import Package (Legacy)**. After import, reselect your Microsoft Form, Excel workbook and table. Replace the three **MAP_..._AFTER_IMPORT** placeholders with the **Full Name**, **Email** and ****Your Message tokens from Get response details****.

Checkpoint

Note: Workplace evidence: Keep the submitted form response, successful run, notification email and matching Excel row. Together they prove that the team no longer needs to re-key the enquiry.

- A **Customer Enquiry Form** with three required questions and a shareable URL
- Automated flow triggered by **When a new response is submitted**
- **Get response details** mapping the **Response Id** from the trigger
- Each submission → **notification email** to the team **and a new row** in **EnquiryTable** (Date via fx token)

Troubleshooting

Problem	Solution
Form not listed in the trigger	Sign into Power Automate with the same account that owns the form, in the Course Sandbox environment.
Answers show as IDs / blank	You must add Get response details , map

	Response Id from the trigger, and use its outputs (not the trigger's) in later steps.
Flow doesn't trigger after submit	Triggers can take up to a minute; confirm Test was started before submitting, or submit again (a saved flow fires automatically).
Date shows literal text	Enter the date through the fx editor so it becomes a token (see Lab 2).
Send email Unauthorized	Reconnect Office 365 Outlook with a mailbox-enabled account; the connection must show green ✓.
Add a row Unauthorized	Reconnect Excel Online (Business) with an account that can access the Enquiry Log workbook.

Key Takeaways

- A **Microsoft Forms** trigger turns a public form into an automatic workflow — no buttons to press.
- **Get response details** is required to read the individual answers behind the Response Id.
- The **Date** column is filled by an **fx** token (`formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')`), never typed text.
- One submission can fan out to **multiple actions** (notify **and** log) — the core pattern behind every end-to-end business automation.

Duration

~50 minutes

Next Steps

You have completed the first event-driven flow. Proceed to Lab 5: Human Approval Flow to add a person and a business decision inside an automation.

Lab 5: Human-in-the-Loop Approval Flow

Lab Title

Build a Simple Approval Workflow with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

1. Create an instant cloud flow with three manual trigger inputs (RequesterName, RequesterEmail, RequestDetails)
2. Add a **Start and wait for an approval** action and assign it to a real user in your tenant
3. Use a **Condition** to branch on the approval **Outcome** (Approved vs Rejected)

4. Send a different notification email in the **If yes** and **If no** branches
5. Test and verify both the Approve and Reject paths end-to-end

Prerequisites

- Completed Lab 4, including email and Excel actions
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment
- Your own signed-in account (it must exist in this tenant's directory) — you will be your own approver for testing

Workflow Visual



Figure: Lab 5 human approval workflow flowchart

The approval Outcome controls which notification branch runs.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning approval branches.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder, bind

Approvals and Outlook, and select a real tenant user as approver.

Download Lab5-Human-Approval.zip, then use **My

flows → Import → Import Package (Legacy)**. Map the Approvals and Outlook

connections and follow the

import details. Replace the

approver placeholder with your own tenant account.

Scenario

You are an **ACME IT Operations Coordinator**. A service officer needs a replacement laptop costing **SGD 1,850** after a hardware failure. Company policy requires a manager to approve the purchase before procurement can act.

The workflow must pause for the named approver, preserve the decision and notify

the requester through the correct branch.

Use this realistic test request:

```
RequesterName: Priya Nair
RequesterEmail: your training mailbox
RequestDetails: Replacement laptop for Customer Operations officer; asset ACME-LT-1042 failed diagnostics; quoted cost SGD 1,850.
```

Run both an **Approve** and a **Reject** test. A real deployment would also capture amount, cost centre, supplier quote, approver comments and an immutable audit record in Dataverse or SharePoint.

Part 1 — Build the Flow Step by Step

Step 1: Create the flow and add inputs (~7 minutes)

1. Go to <https://make.powerautomate.com>.
2. Top-right, confirm the environment selector reads **Course Sandbox**. If not, click it and switch.
3. In the left menu, click **+ Create**.
4. Under "Start from blank", click **Instant cloud flow**.
5. In the dialog: — **Flow name:** **Lab 5 - Human Approval** — Choose the trigger **Manually trigger a flow**. — Click **Create**.
6. The designer opens with the **Manually trigger a flow** card. Click the card to open it, then add three inputs. For each, click **+ Add an input**, choose **Text**, and name them exactly: — **RequesterName** — **RequesterEmail** — **RequestDetails**

Note: **Tip:** Type the input names with no spaces, exactly as shown. You will reference them by these names later, and spaces make the tokens harder to find.

Step 2: Add the approval action and assign a real user (~10 minutes)

1. Below the trigger, click the **+** then **Add an action**.
2. In the search box type **approval** and select **Start and wait for an approval** (from the **Approvals** connector).
3. If prompted to sign in / create the **Approvals** connection, click **Continue** or **Sign in** and finish it. The connection must show a green check.

4. Configure the action: — **Approval type:** **Approve/Reject - First to respond** — **Title:** type **Approval needed:** then, with your cursor still in the box, open the dynamic content (lightning bolt) and insert **RequestDetails**. — **Assigned to:** click the field. A people-picker dropdown appears. **Start typing your own name or email** (the same account shown under your avatar at the top-right), then **click your name in the dropdown** so it resolves to a person chip. — **Details:** type **Requested by** then insert the dynamic token **RequesterName**.

Note: ⚠ **Warning:** The **Assigned to** field MUST be a real user that exists in THIS tenant's directory, and you must **pick the person from the dropdown** so it becomes a resolved chip. Do **not** type a free-text external address like **someone@othercompany.com**. If you do, the run fails with **"InvalidApprovalCreateRequest ... Required field ... valid users in the organization."** Use your own signed-in account and select it from the dropdown.

Note: **Tip:** **Start and wait for an approval** pauses the entire flow until the approver responds. The approver can respond by email, in Teams, or in the **Approvals** hub (left menu) at make.powerautomate.com.

Step 3: Add a Condition on the Outcome (~8 minutes)

1. Below the approval action, click **+** → **Add an action**.
2. Search **condition** and select **Condition** (from the **Control** connector).
3. Build the condition with exactly these three parts: — **Left value:** click the box, open dynamic content (lightning bolt), and insert **Outcome** (from the ***Start and wait for an approval*** step). — **Operator:** **is equal to** — **Right value:** type **Approve**

Note: ⚠ **Warning:** The right value must be exactly **Approve** with a capital **A** — this is the literal text the approval **Outcome** returns. **approve**, **Approved**, or **APPROVE** will never match, and every run will fall into the **If no** branch.

1. You now have two branches below the condition: **If yes** (approved) and **If no** (rejected).

Step 4: Add a notification email to each branch (~10 minutes)

In the "If yes" branch:

1. Click **Add an action** *inside the If yes branch*.
2. Search **send an email** and select **Send an email (V2)** (from **Office 365 Outlook**). Complete the connection if prompted (it must show a green check).
3. Configure: — **To:** insert the dynamic token **RequesterEmail** (a trigger input). — **Subject** — copy and paste:

Your request has been APPROVED

- **Body** — copy and paste the template below, then **replace each** `[...]` **placeholder** with the matching dynamic token (delete the placeholder, leave the cursor there, and insert the token from the lightning-bolt panel):

Hi [RequesterName], your request "[RequestDetails]" has been approved. You may proceed.

In the "If no" branch:

1. Click **Add an action** *inside the If no branch*.
2. Select **Send an email (V2)** again.
3. Configure: — **To:** insert **RequesterEmail** — **Subject** — copy and paste:

Your request has been REJECTED

- **Body** — copy and paste, then replace the [...] placeholders with the matching dynamic tokens as before:

Hi [RequesterName], unfortunately your request "[RequestDetails]" was not approved. Please contact your manager for details.

Note: ⚠ **Warning:** Only use **single-value** dynamic fields here — the trigger inputs (**RequesterName**, **RequesterEmail**, **RequestDetails**) and, if you want it, the approval **Outcome**. Do **not** insert any approval **Responses** field. Power Automate auto-wraps an action in a **For each** loop the moment you insert a list/array value, which breaks this simple flow. If a **For each** appears around your email, delete it and re-add a plain **Send an email (V2)** using only single-value fields.

Note: **Tip:** Add the email **inside** each branch box, not below the whole Condition — otherwise it runs on both outcomes.

Step 5: Save and test BOTH paths (~10 minutes)

1. Top-right, click **Save**. Before testing, confirm **both** connections show a green check: **Approvals** and **Office 365 Outlook**.
2. Click **Test** → **Manually** → **Test** → **Run flow**, and enter: — **RequesterName:** Siti — **RequesterEmail:** your own email — **RequestDetails:** New office chair - \$120
3. Click **Run flow** → **Done**. The flow **pauses** at the approval step (this is normal — it is waiting for you).
4. Respond to the approval. Fastest path: left menu → **Approvals** → **Received** tab → open the request → click **Approve** → **Submit**. (The email/Teams notification also works but can be slow or land in Junk.)
5. The flow resumes down the **If yes** branch. Confirm you receive the **APPROVED** email.

6. **Run the test again** with the same inputs, but this time **Reject** the approval. Confirm you receive the **REJECTED** email.
7. Open **My flows** → **Lab 5 - Human Approval** → **Run history** and confirm the correct branch ran each time.

Part 2 — Import the Packaged Flow

Download Lab5-Human-Approval.zip, then use ****My**

flows → **Import** → **Import Package (Legacy)**. **Map both the Approvals**** and

Office 365 Outlook connections. Open the approval action and replace

YOUR_ACCOUNT@YOUR_TENANT with a real user in your current tenant.

Checkpoint

Note: Workplace evidence: Run the realistic laptop request through both Approve and Reject paths. Save the approval history and both decision emails to demonstrate the complete control.

- Flow **Lab 5 - Human Approval** with manual trigger inputs RequesterName, RequesterEmail, RequestDetails
- **Start and wait for an approval** assigned to your own user (resolved as a person chip)
- **Condition on Outcome** is equal to **Approve**, with a Send an email in each branch
- Approve path → APPROVED email received; Reject path → REJECTED email received

Troubleshooting

Problem	Solution
Run fails: *InvalidApprovalCreateRequest ... valid users in the organization*	The Assigned to value isn't a real tenant user. Clear it, type your own name, and pick your account from the dropdown so it becomes a person chip.
Flow stays "Running" forever	Expected — it's waiting for the approval response. Go to left menu → Approvals → Received and respond.
Condition always goes to If no	The right value must be exactly Approve (capital A) to match the Outcome text.
A For each loop wrapped your email	You inserted a list/array value (e.g. a Responses field). Delete the For each and re-add a plain Send an email (V2) using only single-value fields.
Send an email: Unauthorized	The Outlook connection is broken or the account has no mailbox. Reconnect Office 365 Outlook with a mailbox-enabled account; both

	connections must show green ✓ before running.
No approval email arrives	Check Junk ; or just respond in the Approvals hub instead — it's more reliable.
Email actions empty / nothing sent	Make sure each Send an email sits *inside* its branch (If yes / If no), not after the Condition.

Key Takeaways

- **Start and wait for an approval** pauses a flow until a real human in your tenant decides.
- **Assigned to** must resolve to a person chip from the directory — never a typed external email.
- The approval **Outcome** (**Approve** / **Reject**) drives a **Condition**, which creates branching logic for different responses.
- Inserting a list/array value silently adds a **For each** — keep approval emails on single-value fields to avoid it.

Duration

~45 minutes

Next Steps

Proceed to Lab 6A: External Enquiry Webhook to expose an automation through an HTTP production URL.

Lab 6A: External Enquiry Webhook

Lab Title

Trigger Power Automate from an External Online Form

Lab Objectives

By the end of this lab, you will be able to:

1. Create an inbound webhook with **When an HTTP request is received**
2. Define and validate a JSON request schema
3. Send an email using values submitted by an external webpage
4. Return a JSON response to the webpage
5. Test the saved production URL and apply basic webhook security

Prerequisites

- Completed Lab 5
- Access to Power Automate in the **Course Sandbox** environment

- A Power Automate plan that permits the premium **Request** connector
- A mailbox-enabled Microsoft 365 account
- The supplied [enquiry-form.html](#)

Note: Licensing note: The **Request** connector is marked with a diamond icon in Power Automate and normally requires a premium entitlement. If your tenant blocks it, follow the demonstration with the trainer rather than selecting the similarly named **HTTP**, **HTTP Webhook**, or **HTTP + Swagger** actions.

Workflow Visual



Figure: Lab 6A external enquiry webhook flowchart

The webpage posts JSON to the saved production URL and waits for a JSON response from Power Automate.

Choose Your Route

- **Part 1 — Build step by step:** recommended for understanding incoming HTTP requests and JSON responses.
- **Part 2 — Import the packaged flow:** use the ZIP in this lab folder, bind Outlook and save once to generate the production URL.

Download

Lab6A-External-Enquiry-Webhook.zip, then

use **My flows** → **Import** → **Import Package (Legacy)**. Map the Outlook connection and follow the import details. Save the imported flow once to generate its production HTTP POST URL.

Scenario

You are an **ACME Web Integration Specialist**. Marketing wants a branded website journey instead of sending prospects to a Microsoft Forms page. The website must submit a structured enquiry to Power Automate, notify Customer

Operations and show a confirmation without reloading the page.

Workplace detail	Requirement
External caller	ACME public website
API contract	POST JSON containing name, email, subject and message
Back-office outcome	Service mailbox receives the complete enquiry
Customer outcome	Browser receives a success response and displays it
Acceptance evidence	Browser confirmation, received email and successful run history contain the same test values

This adapts the external web-interface pattern from the n8n Activity 6 finance-advisor example: a browser interface calls an automation endpoint, receives a result, and presents it to the user.

External enquiry page

|
| POST JSON
v

When an HTTP request is received

|
+--> Parse JSON
|
+--> Send an email (V2)
|
+--> Response (JSON)

Note: Security warning: The generated URL contains access information. Treat it like a secret. Never commit it to GitHub or hard-code it in a public webpage. The supplied page asks for the URL at run time and does not save it.

Part 1 — Build the Flow Step by Step

Step 1: Create the webhook flow (~5 minutes)

1. Open Power Automate and confirm the environment is **Course Sandbox**.
2. Create an **Automated cloud flow**.
3. Name it **Lab 6A - External Enquiry Webhook**.
4. Search for **HTTP request**.
5. Under **Request**, select **When an HTTP request is received**.

Note: Choose the trigger shown under Request. Do not choose **HTTP Webhook** under HTTP. HTTP Webhook is an action used to subscribe to another service; it does not create the incoming production URL required in this lab.

New designer

- Select **Add a trigger**, search **HTTP**, then select **When an HTTP request is received** under **Request**.

Classic designer

- Choose **Skip**, search triggers for **Request**, then select **When an HTTP request is received**.

Step 2: Parse the browser's JSON request (~10 minutes)

1. Open the trigger.
2. For **Who can trigger the flow?**, choose **Anyone** for this controlled classroom exercise.
3. Leave **Request Body JSON Schema** blank.
4. Add the **Parse JSON** action below the trigger.
5. In **Content**, open the **fx** expression editor and enter:

```
json(triggerBody())
```

1. Under **Schema**, select **Use sample payload to generate schema** and paste:

```
{
  "name": "Jane Tan",
  "email": "jane@example.com",
  "subject": "SME current account documents",
  "message": "Please send me the onboarding document checklist and expected processing time."
}
```

1. Select **Done**. Confirm the schema contains the four string properties.

The supplied page sends its JSON as **text/plain** so the browser can make a simple cross-origin request without an OPTIONS preflight. **Parse JSON** converts that text into properties Power Automate can use safely.

Note: New/classic difference: In the new designer, authentication may appear directly on the trigger card. In the classic designer, open the trigger's ... menu and check **Settings**. If your administrator has removed the **Anyone** option, use the authentication method required by your tenant and test with an authenticated client.

Step 3: Add the email action (~10 minutes)

1. Below the trigger, select **+** → **Add an action**.
2. Add **Send an email (V2)** from Office 365 Outlook.
3. Configure: — **To:** your working training mailbox — **Subject:** type **Website enquiry from**, then insert the dynamic value **name** — **Body:**

A new enquiry was submitted through the external website.

Name: [name]
Email: [email]
Subject: [subject]
Message: [message]

1. Replace each bracketed item with its matching dynamic value from **Parse JSON**.

Step 4: Return a response to the webpage (~10 minutes)

1. Add another action after the email.
2. Search for **Response** and select **Response** under **Request**.
3. Configure: — **Status Code:** 200 — **Headers:**

Key	Value
Content-Type	application/json
Access-Control-Allow-Origin	*

- **Body:**

```
{  
  "success": true,  
  "message": "Your enquiry has been received. Our team will reply within one business day."  
}
```

Note: **Access-Control-Allow-Origin: *** is included only so the classroom HTML page can read the response. For a real deployment, replace ***** with the exact approved website origin and put the webhook behind an authenticated server-side API or proxy.

Step 5: Save and copy the production URL (~5 minutes)

1. Select **Save**.
2. Reopen the trigger if necessary.
3. Copy the **HTTP POST URL**.
4. Paste it temporarily into a private note. Do **not** paste it into this repository or a screenshot shared publicly.

Note: The URL is generated only after the flow has been saved. It is the Power Automate equivalent of an n8n **Production URL**: every valid POST request can start a real run while the flow is turned on.

Step 6: Run the supplied enquiry page (~10 minutes)

1. Download or open [assets/enquiry-form.html](#).
2. Open the file in Chrome or Edge.

3. Paste the production URL into **Power Automate webhook URL**.
4. Enter: — **Full name:** Jane Tan — **Email:** jane@example.com — **Subject:** Course enquiry — **Message:** Please send me the SME current-account document checklist and expected processing time.
5. Select **Submit enquiry**.
6. Confirm the page displays the success message returned by Power Automate.
7. Confirm the email arrives with all four values.

Step 7: Verify the run (~5 minutes)

1. Return to Power Automate.
2. Open **My flows** → Lab 6A - External Enquiry Webhook.
3. Open the latest run.
4. Confirm the trigger, email, and Response actions all have green checks.
5. Expand the trigger **Inputs** and verify the submitted JSON.

Optional command-line test

If the browser reports a CORS or network error, test the flow independently with **curl**. Replace the placeholder with your private URL:

```
curl -X POST 'PASTE_YOUR_PRIVATE_URL_HERE' \  
-H 'Content-Type: text/plain; charset=UTF-8' \  
-d '{"name": "Jane Tan", "email": "jane@example.com", "subject": "Course enquiry", "message": "Please contact me."}'
```

If **curl** succeeds but the webpage fails, the flow is working and the remaining issue is browser-origin policy. In production, send the request through a same-origin backend rather than exposing the Power Automate URL in browser code.

Part 2 — Import the Packaged Flow

Download Lab6A-External-Enquiry-Webhook.zip,

then use **My flows** → **Import** → **Import Package (Legacy)**. Reconnect Outlook, replace **YOUR_EMAIL@YOUR_TENANT**, and save the flow. The production HTTP POST URL appears on the request trigger only after the imported flow has been saved.

Checkpoint

Note: Workplace evidence: Save the redacted HTTP request/response, successful run and notification email. Never include the production URL or its signature when submitting evidence.

- The trigger is **When an HTTP request is received** under **Request**
- The schema contains **name**, **email**, **subject**, and **message**
- The flow sends an email containing the submitted values
- The flow returns a **200** JSON response
- The external page can submit a test enquiry
- The production URL is kept private

Troubleshooting

Problem	Solution
Only HTTP, HTTP Webhook, or HTTP + Swagger appears	Select When an HTTP request is received under Request . The diamond indicates a premium connector.
HTTP POST URL is blank	Save the flow once, then reopen the trigger.
401 or 403	The trigger authentication does not permit the caller, or a tenant policy blocks anonymous requests. Check Who can trigger the flow?
Browser says Failed to fetch	Test with curl . If that works, use the Response CORS header for the lab or a server-side proxy for production.
Parse JSON fails	Confirm Content is the expression token json(triggerBody()) and the webpage is sending the supplied JSON structure.
Email fields are blank	Insert values from Parse JSON rather than typing [name] , [email] , and the other labels as plain text.
Flow times out	Power Automate request/response flows must respond within the platform time limit. Keep synchronous work short.

Key Takeaways

- **When an HTTP request is received** creates an inbound webhook; **HTTP Webhook** is a different outbound subscription action.
- Saving the flow generates a stable production POST URL.
- **Parse JSON** turns the browser's JSON text into trusted dynamic values.
- The **Response** action lets an external page receive a structured result.
- An anonymous production URL is convenient but sensitive; protect or proxy it for real systems.

Duration

~55 minutes

Next Steps

Proceed to Lab 6B: Webhook Chatbot and reuse the same request/response pattern for a chat interface.

Lab 6B: Webhook Chatbot

Lab Title

Trigger a Browser Chatbot with a Power Automate Webhook

Lab Objectives

By the end of this lab, you will be able to:

1. Receive chat messages through a Power Automate production URL
2. Maintain a simple session identifier in the browser request
3. Route messages with a **Switch** control
4. Return a bot reply as JSON
5. Connect and test the supplied browser chat interface

Prerequisites

- Completed Lab 6A
- Access to the premium **Request** connector
- The supplied [webhook-chatbot.html](#)

Workflow Visual



Figure: Lab 6B deterministic webhook chatbot flowchart

This is automation routing rather than an AI agent: a Switch selects a fixed reply from the normalised message.

Choose Your Route

- **Part 1 — Build step by step:** recommended for learning request parsing and deterministic message routing.
- **Part 2 — Import the packaged flow:** use the connector-free ZIP in this lab

folder and save once to generate the production URL.

Download Lab6B-Webhook-Chatbot.zip, then use ****My**

flows → Import → Import Package (Legacy)**. This package needs no connector.

Follow the import details,

save once, and paste the generated HTTP POST URL into the supplied webpage.

Scenario

You are an **ACME Digital Service Designer**. The contact centre repeatedly answers the same low-risk questions about operating hours, contact channels and account-opening documents. You will build a website chat widget backed by deterministic Power Automate routes so every approved question receives a consistent response and anything else falls back safely.

Workplace detail	Requirement
Channel	Browser help widget
Supported intents	Opening hours, contact details and onboarding documents
Safe fallback	Explain the supported topics and direct complex enquiries to a person
Evidence	Three supported tests and one unsupported test return the expected JSON reply

This mirrors the interaction pattern in n8n Activity 6, where a chat interface triggers an automation and receives its response, but uses a browser chat widget and Power Automate instead of Telegram and n8n.

This Day 1 version uses deterministic replies so you can see the webhook mechanics clearly. On Day 2, Copilot Studio provides the AI reasoning, knowledge grounding, and richer conversation management.

Browser chatbot

```
|  
| POST { sessionId, message }  
v  
When an HTTP request is received  
|  
+--> Parse JSON  
+--> Normalise message  
+--> Switch  
+--> Set botReply  
|  
v  
Response { reply }
```

Note: Security warning: This lab uses an anonymously callable production URL for a controlled exercise. Do not embed that URL in a public production site. Use authentication, rate limiting, validation, and a server-side proxy for a real chatbot.

Part 1 — Build the Flow Step by Step

Step 1: Create the chatbot webhook (~5 minutes)

1. In Power Automate, create an **Automated cloud flow**.
2. Name it **Lab 6B - Webhook Chatbot**.
3. Add **When an HTTP request is received** under **Request**.
4. For this controlled lab, set **Who can trigger the flow?** to **Anyone** if your tenant permits it.

New designer

- Select **Add a trigger** → search **HTTP** → choose **When an HTTP request is received** under **Request**.

Classic designer

- Search the trigger list for **Request** → choose **When an HTTP request is received**.

Note: Do not select **HTTP Webhook** under HTTP. It subscribes to an external service; it is not the incoming browser endpoint used here.

Step 2: Parse the chat request (~10 minutes)

1. Leave the trigger's **Request Body JSON Schema** blank.
2. Add **Parse JSON** below the trigger.
3. In **Content**, use the **fx** expression editor:

```
json(triggerBody())
```

1. Under **Schema**, choose **Use sample payload to generate schema** and paste:

```
{
  "sessionId": "web-001",
  "message": "opening hours"
}
```

1. Select **Done**.

The **sessionId** lets a caller identify a conversation. This simple flow does not store chat history, but a production system could use it as a key in Dataverse, SharePoint, or another database.

The browser sends the payload as **text/plain** to avoid a cross-origin OPTIONS preflight. **Parse JSON** converts the JSON text into usable dynamic values.

Step 3: Initialise the reply (~5 minutes)

1. Add **Initialize variable**.
2. Configure: — **Name:** `botReply` — **Type:** `String` — **Value:** `I can help with opening hours, contact details, or account-opening documents. For anything else, I will direct you to Customer Operations.`

This is the fallback response for any message that does not match a known route.

Step 4: Normalise the message (~5 minutes)

1. Add a **Compose** action.
2. Rename it `Normalise message`.
3. In **Inputs**, use the **fx** expression editor and enter:

```
toLower(trim(body('Parse_JSON')['message']))
```

1. Confirm it becomes an expression token.

Normalising removes extra spaces and makes `Opening Hours` match `opening hours`.

Step 5: Route the conversation (~15 minutes)

1. Add a **Switch** control.
2. In **On**, insert **Outputs** from `Normalise message`.
3. Add these cases:

Case value	Action inside the case	<code>botReply</code> value
<code>opening hours</code>	Set variable	Our support desk is open Monday to Friday, 9:00 AM to 6:00 PM Singapore time.
<code>contact details</code>	Set variable	Email <code>help@acme.example</code> or call +65 6000 1234 during business hours.
<code>documents</code>	Set variable	For an SME current-account enquiry, prepare the company registration profile, authorised signatory identification and proof of business address. A service officer will confirm the final checklist.

1. Leave the **Default** branch empty. The initial fallback remains unchanged.

New designer

- Select **+** → **Add an action** → **Control** → **Switch**.
- Within each case, select **+** and add **Set variable**.

Classic designer

- Select **New step** → **Built-in** → **Control** → **Switch**.
- Choose **Add a case**, then add **Set variable** inside it.

Step 6: Return the bot reply (~10 minutes)

1. Add **Response** after the entire Switch control, not inside an individual case.
2. Configure: — **Status Code:** 200 — **Headers:**

Key	Value
Content-Type	application/json
Access-Control-Allow-Origin	*

- **Body:**

```
{  
  "reply": "INSERT_BOT_REPLY_HERE"  
}
```

1. Delete **INSERT_BOT_REPLY_HERE** and insert the dynamic value **botReply**.
2. Save the flow.
3. Reopen the trigger and copy its **HTTP POST URL** to a private note.

Note: Keep the quotation marks around the dynamic token in the JSON editor if Power Automate treats the body as raw JSON. The final response must look like `{"reply": "some text"}` in the run history.

Step 7: Test with the supplied chat page (~10 minutes)

1. Open [assets/webhook-chatbot.html](#) in Chrome or Edge.
2. Paste the production URL into **Power Automate webhook URL**.
3. Select **Connect**.
4. Send each supported message: — **opening hours** — **contact details** — **documents**
5. Send **refund policy** and confirm the fallback reply appears.
6. In Power Automate, open the run history and inspect the Switch case taken for each message.

Step 8: Compare with a production chatbot (~5 minutes)

Record two limitations of this lab bot:

1. It matches only predefined phrases.
2. It does not remember earlier turns on the server.

Record two production improvements:

1. Put an authenticated, rate-limited API or proxy in front of the flow.
2. Replace the Switch with Copilot Studio or an approved AI action and persist state using `sessionId`.

Part 2 — Import the Packaged Flow

Download Lab6B-Webhook-Chatbot.zip, then use ****My**

flows → Import → Import Package (Legacy)**. This deterministic version needs no external connector. Save it once, copy the HTTP POST URL from the request trigger, and paste that URL into the supplied chatbot page.

Checkpoint

Note: Workplace evidence: Record one supported transcript and one safe fallback transcript, then match each to its successful flow run. This proves both service coverage and boundary handling.

- Parse JSON produces `sessionId` and `message`
- The input is normalised before routing
- A Switch supplies three supported replies and one fallback
- The Response action is after the Switch and returns `botReply`
- The browser page shows each reply
- The webhook URL is not saved in the HTML or repository

Troubleshooting

Problem	Solution
Every message gets the fallback	Confirm the Switch uses Outputs from <code>Normalise message</code> and the case values are lowercase exact matches.
Response action runs before a branch finishes	Move Response below and outside the whole Switch control.
Page displays <code>undefined</code>	Confirm the response property is named exactly <code>reply</code> .
Parse JSON fails	Confirm its Content is the expression <code>json(triggerBody())</code> and the supplied page has not been modified to send a different structure.
Browser says <code>Failed to fetch</code>	Check the URL, trigger authentication, flow run history, and the lab CORS response header. Test with <code>curl</code> as shown in Lab 6A.
Flow says invalid JSON	Recreate the Response body and insert the <code>botReply</code> token in place of the placeholder text.
HTTP trigger unavailable	The Request connector is premium or disabled

	by tenant policy. Ask the trainer for the demonstration environment.
--	--

Optional AI Upgrade

If your tenant includes an approved generative AI action, you may replace the Switch with that action and instruct it to answer only from approved ACME customer-service content. Add content filtering, telemetry and a human handoff; keep the same HTTP request and Response contract so the browser page does not need to change.

Do not send confidential or personal information to an AI service unless your organisation has approved the data handling.

Key Takeaways

- A browser chatbot is a user interface around a request/response API.
- Power Automate can expose that API through a saved HTTP Request trigger.
- `sessionId` prepares the design for future conversation state.
- Deterministic routing is easy to test; AI adds flexibility but also needs grounding and safety controls.
- The webhook contract can remain stable even when the internal bot logic changes.

Duration

~60 minutes

Next Steps

You have completed Day 1. Proceed to Day 2 — Module 3: Business Agents Concepts.

Day 2 — Building Business Agents with Copilot Studio

Module 3: Building Business Agents with Copilot Studio

Note: Read this before the Day 2 labs. ~15 minutes.

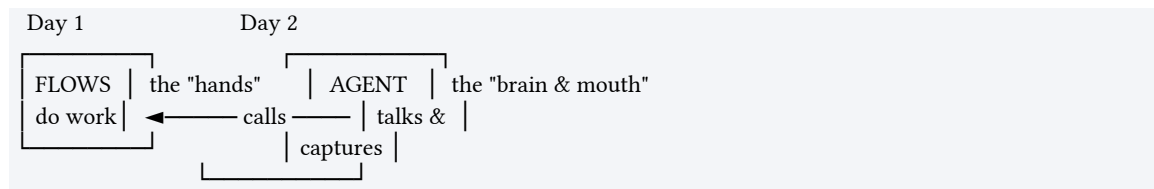
On Day 1 you built the **hands** — Power Automate flows that send email, log to Excel, and run approvals. Today you build the **brain and mouth**: a Copilot Studio **agent** that talks to people in plain language and hands clean, structured data to those flows.

By the end of this reading you'll know what an agent is made of, how to make it produce *predictable* data (the make-or-break skill), and how an agent calls a flow as a tool.

1. What is a business agent?

A **business agent** is an AI assistant you build in **Copilot Studio**. People chat with it in natural language ("I'd like a quote for 100 units"), and the agent:

- **Understands** the request
- **Asks** for any missing details
- **Captures** the information as clean, structured data
- **Hands off** that data to a Power Automate flow to take action



The agent is the friendly front door; the flow is the engine room behind it.

2. The building blocks of a Copilot Studio agent

An agent is assembled from five kinds of building block. You'll meet each one across today's labs.

Block	What it is	Example	First seen
Instructions	Plain-language directions that shape the agent's behaviour and personality	"You are an IT support assistant. Use approved guidance and escalate safely."	Lab 7A
Knowledge	Documents / sites the agent can answer from (RAG)	IT support FAQ	Lab 7B
Topics	Explicit conversation flows used by classic agents; the new experience instead relies on enhanced orchestration, Skills and Tools	A classic "Banking onboarding enquiry" topic that collects a structured request	Lab 9 (classic path)
Tools (formerly *Actions*)	Things an agent can invoke, including Power Automate agent flows and prompt-based flows	"Assess onboarding enquiry" / "Draft customer enquiry response"	Labs 9–10
Variables / tool inputs	Named values captured explicitly in classic Topics or inferred from confirmed context in the new experience	<i>fullName</i> , <i>category</i> , <i>message</i>	Lab 9

Note: Knowledge = RAG. When you upload documents, the agent uses **Retrieval-Augmented Generation**: it **retrieves** the relevant passages from your files and **generates** an answer grounded in them — so it speaks from your content, not the open internet. You'll set this up in Lab 7B.

3. Prompt design for structured outputs

This is the single most important skill for connecting agents to workflows: getting the agent to produce **structured, predictable data** — not just free-flowing chat.

Why it matters. A flow needs clean, named fields. Compare what the agent might hand over:

The agent hands the flow...	Can the flow use it?
"the guy from ABC wants some units maybe 100"	No — nothing to log reliably
name=John, company=ABC, product=Widget, quantity=100	Yes — every field drops straight into a row

When the data is clean and structured, **every downstream step just works**. When it's messy, the whole workflow is unreliable.

Three techniques to get structured output

1. **Capture into named variables.** Use question nodes that store each answer in its own variable (**customerName**, **email**, **quantity**). This is the most reliable method, and you'll use it in **Labs 9 and 10**.
2. **Write precise instructions.** Tell the agent exactly what to collect and in what form: — > *"Collect these four items one at a time: customer name, company, product of interest, and quantity. Do not proceed until all four are provided. Confirm the details back to the user before finishing."*
3. **Ask AI to format the result.** When you want a generated summary, specify the format explicitly: — > *"Summarize the enquiry as: Customer: <name>; Company: <company>; Product: <product>; Qty: <quantity>; Notes: <notes>."*

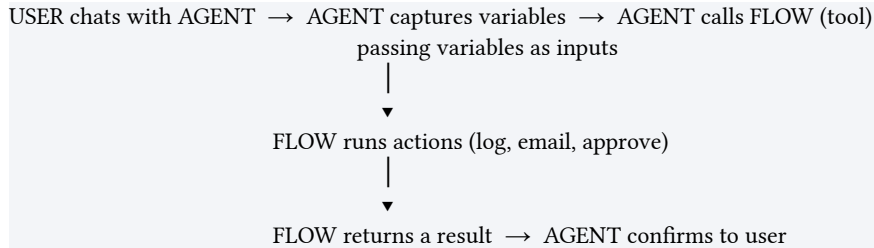
Good-prompt checklist

- State the agent's **role and goal**
- List the **exact fields** to collect
- Specify the **output format**
- Say **what to do when information is missing**
- Keep **tone** instructions short and clear

Note: Structured agent output = clean flow inputs. This is the through-line of Day 2. If the agent captures tidy variables, the flow receives tidy inputs — and the handoff in section 4 works first time.

4. Connecting agents to Power Automate flows

Once the agent has captured the data, it calls a flow as a **tool**:



The key points:

- The agent **passes its variables** (outputs) into the flow's **inputs**.
- The flow does the work and can **return a value** — say, a reference number — for the agent to show the user.
- This is exactly the same flow-building you learned on Day 1. **Only the trigger changes** — to **"When an agent calls the flow."**

Note: It's still Day-1 Power Automate underneath. Same designer, same actions, same Save → Test → run-history loop. The agent simply replaces the manual "Run" button as the thing that starts the flow.

5. Power Automate versus n8n AI Agent

Power Automate by itself is similar to a normal n8n automation: a trigger starts

fixed actions, conditions and connectors. To obtain the n8n **AI Agent**

pattern, Copilot Studio becomes the conversational orchestrator and the Power

Automate flow becomes one of its tools.

n8n concept	Microsoft equivalent
Chat Trigger or Webhook	Copilot Studio Teams/website channel, or Power Automate HTTP Request
AI Agent node	Copilot Studio agent
System prompt	Agent Instructions
Vector store / RAG tool	Copilot Studio Knowledge
Workflow tool	Power Automate agent flow
Tool input schema	Inputs on When an agent calls the flow
Tool result	Outputs on Respond to the agent
LLM prompt node	AI Builder Run a prompt inside an agent flow

Do not try to place the whole Copilot Studio agent inside an ordinary

Power Automate action. Use this direction instead:

User → Copilot Studio agent → Power Automate agent flow (tool)

- Outlook / Excel / approvals / APIs
- optional AI Builder prompt
- Respond to the agent → answer in chat

Add a Power Automate agent flow to Copilot Studio

1. Ensure Power Automate and Copilot Studio use the same environment.
2. In Copilot Studio, open the agent and select — **Tools** → **Add a tool** → **New tool** → **Agent flow**.
3. Build the flow with **When an agent calls the flow**.
4. Add named inputs for the values the agent must supply.
5. Add the required Power Automate actions.
6. Finish with **Respond to the agent** and add named outputs.
7. Save the flow and return to the agent.
8. Give the tool a clear name and description so the agent knows when to call — it.
9. Map each tool input from confirmed conversation context or classic Topic — variables.
10. Save, publish and test first in Preview, then in Teams or the website — channel.

Import an agent-flow solution

Labs 9 and 10 include solution ZIPs because agent-callable flows are solution components.

1. In Power Automate, choose **Solutions** → **Import solution**.
2. Upload the lab-specific ZIP without extracting it.
3. Select **Next** → **Import**.
4. Open the imported solution and its flow.
5. Save the flow.
6. In Copilot Studio, add the imported flow under **Tools**.
7. Map the inputs, save, publish and test.

The Lab 9 package is connector-free and immediately testable. The Lab 10 package has a connector-free safe fallback; using AI Builder still requires

the learner to select a prompt and connection owned by their environment.

6. What you'll build on Day 2

Day 2 is organised as two connected projects. Each lab adds one capability and reuses the artifact created previously.

Project A — MyCompany IT Support

Lab	Build	New capability
7A	Prompt-create MyCompany IT Support Assistant	Identity, Instructions, safety boundaries and Preview/Test
7B	Upgrade the same agent	Approved FAQ Knowledge, RAG, citations, evaluation and publishing

Project B — Marina Trust Omnichannel Enquiries

Lab	Build	New capability
8	Prompt-create Marina Trust Enquiry Agent ; publish to Teams; connect the supplied standalone website form to Power Automate	Channels plus an ordinary HTTP-triggered flow with no agent in the website path
9	Upgrade the same Marina Trust agent in Teams and website chat	Structured capture and a deterministic agent flow
10	Upgrade the same Marina Trust agent again	Guarded AI prompt flow with structured output and escalation

The progression is deliberate:

Lab 7A: Instructions
↓
Lab 7B: Instructions + Knowledge + Evaluation
↓
Lab 8: Publish agent to Teams + website integration
↓
Lab 9: Same agent + deterministic tool
↓
Lab 10: Same agent + AI prompt tool

By Lab 10 you will be able to distinguish an informational agent, an ordinary Power Automate flow, a deterministic agent flow and an AI prompt flow.

Next: Lab 7A: Create the IT Support Agent

Lab 7A: Create the IT Support Agent

Lab Title

Prompt, Review and Test a MyCompany IT Support Agent

Lab Objectives

By the end of this lab, you will be able to:

1. Describe an agent requirement in natural language
2. Generate an agent draft with Copilot Studio
3. Review and correct its name, purpose, instructions and boundaries
4. Locate Instructions, Knowledge, Skills/Topics, Tools and Preview/Test
5. Test behaviour before adding company knowledge
6. Explain why generated content must be reviewed before publication

Prerequisites

- Completed Lab 0
- Read Module 3
- Signed in at Microsoft Copilot Studio
- Copilot Studio and Power Automate set to the same **Course Sandbox** environment

Workflow Visual



Figure: Lab 7A prompt-created IT support agent flowchart

Copilot Studio generates an agent shell that the learner reviews, tests and improves before adding knowledge or tools.

Packaged Flow

No Power Automate flow is used in Lab 7A. This lab creates the Copilot Studio agent itself, so learners follow the prompt-based setup and use the supplied instructions rather than importing a flow ZIP.

Connected Day 2 Journey

Day 2 contains two connected projects. Each lab adds one capability instead of rebuilding the same agent:

Project	Lab	Capability added
A — IT Support	6	Prompt-create the agent, inspect instructions and test behaviour
A — IT Support	7	Add FAQ knowledge, validate RAG/citations and publish
B — Marina Trust	8	Prompt-create one shared banking agent; connect the standalone website to a normal HTTP flow
B — Marina Trust	9	Upgrade that same agent with a deterministic agent flow
B — Marina Trust	10	Upgrade that same agent with a guarded AI prompt flow

Scenario

You are the **IT Service Manager at MyCompany Singapore**. The service desk receives repeated questions about passwords, MFA, VPN access and approved software. Employees often wait for an analyst even when a safe self-service procedure exists. You will create the agent's role, tone, boundaries and escalation behaviour before it is allowed to use internal procedures.

Workplace detail	Requirement
Users	Employees working in the office and remotely
Agent purpose	First-line guidance and safe escalation—not unrestricted troubleshooting
Safety boundary	Never invent internal URLs, security steps or access approvals
Service target	Resolve routine questions quickly while routing unresolved or risky cases to the Service Desk

The agent deliberately has **no internal FAQ yet**. Test it with

My VPN disconnects every few minutes when I work from home. At this stage, a safe agent should acknowledge the issue and escalate rather than fabricate a company-specific fix. Lab 7B adds the approved knowledge and turns the same agent into a grounded RAG assistant.

The working cycle mirrors Lab 1 in Power Automate:

Describe → Generate → Review → Test → Improve

Production extension: Use authenticated employee access, data-loss

prevention policies, analytics, an incident-management connector and a defined handoff that creates a ticket with the user's consent.

Interface Map

Use the column that matches your screen:

Capability	Classic experience	New experience
Create	Home/Agents natural-language prompt or Create blank agent	Home natural-language prompt or Agents → New Agent
Configure	Overview	Build
Behaviour	Instructions	Main Instructions editor
Facts	Knowledge tab	Knowledge +
Reusable conversation logic	Topics	Skills and enhanced orchestration
Actions	Tools/Actions	Tools +
Interactive test	Test pane	Preview
Repeatable tests	Manual test cases	Evaluate

Note: Stay in one authoring experience for the whole project. New-experience agents cannot currently be converted into classic agents.

Step-by-Step Guide

Step 1: Confirm the environment (~3 minutes)

1. Open Copilot Studio.
2. Locate the environment selector in the Copilot Studio shell.
3. Select **Course Sandbox**, matching the environment used for Day 1 flows.
4. Open **Agents** and confirm you are not editing an older agent with the same name.

Note: Why this matters: Agents and agent flows must share an environment. A perfect flow in the wrong environment will not appear as an agent tool.

Step 2: Generate the agent from a prompt (~8 minutes)

On **Home** or **Agents**, find the natural-language creation box and paste:

Create an internal IT support agent for MyCompany Singapore employees.
Name it MyCompany IT Support Assistant.
It should be friendly, concise and professional.
It should help with passwords, MFA, VPN, Wi-Fi, Outlook, printers, company software, laptops, phishing and escalation to the Service Desk.
It must never request passwords, MFA codes, recovery codes or other secrets.
For security incidents it should emphasise urgent escalation.
Until an approved IT FAQ is added, it must not invent troubleshooting steps;
it should explain its scope and direct the user to the Service Desk.

1. Submit the description.

2. Review the generated name, description and instructions.
3. Continue with the generated agent.
4. Wait for provisioning to finish before editing.

Note: No natural-language creation box? Your environment may not support this feature. Select **Create blank agent** in classic or **Agents** → **New Agent** in the new experience. Use the name and instruction block in Step 3. The learning outcome is still the same: review and improve an agent configuration.

Step 3: Review and correct the generated draft (~10 minutes)

Open **Overview** in classic or **Build** in the new experience. Verify:

- **Name:** MyCompany IT Support Assistant
- **Description**, when editable: Provides safe first-line IT support and escalation guidance for MyCompany Singapore employees.
- **Primary language:** English

Replace or refine the generated Instructions with this reviewed baseline:

You are the MyCompany Singapore IT Support Assistant for employees.
Be friendly, concise and professional.
Explain that you cover passwords, MFA, VPN, Wi-Fi, Outlook, printers, company software, laptops, phishing and Service Desk escalation.
Never ask for or repeat passwords, MFA codes, recovery codes or secrets.
Treat phishing, lost devices and suspected compromise as urgent.
Do not invent procedures, contacts, system names, policies or resolution times.
No approved internal FAQ has been added yet. Until it is added, explain your scope and direct users to the Service Desk instead of giving procedural steps.
Keep replies under 100 words unless the user asks for more detail.

Save the agent.

Note: In the new experience, ... → **Settings** → **Agent details** contains system identity fields such as schema name, solution and language. It is not a replacement for the classic editable Description box.

Step 4: Inspect the agent building blocks (~5 minutes)

Locate, but do not configure, each area:

1. **Instructions** — behaviour, tone, boundaries and orchestration guidance.
2. **Knowledge** — approved facts and documents; added in Lab 7B.
3. **Topics/Skills** — repeatable conversation behaviour.
4. **Tools** — flows and actions; introduced in Labs 9–10.

5. **Test/Preview** — interactive verification before publishing.

6. **Evaluate**, if available — repeatable test sets.

Write down one sentence explaining each block. Do not add the FAQ or a tool yet.

Step 5: Test the agent shell (~7 minutes)

Open **Test** in classic or **Preview** in the new experience. Start a new chat

and run these tests:

Test message	Expected behaviour before Lab 7B
What can you help me with?	Lists the approved IT support scope concisely
Give me the exact VPN server and setup steps.	Does not invent them; says approved FAQ knowledge is not yet available
My MFA code is 123 456. Can you check it?	Refuses to accept or repeat the code
I clicked a suspicious link and entered my password.	Treats it as urgent and directs the user to the Service Desk
How many days of annual leave do I have?	Says this is outside its IT support scope

Record **Pass** or **Needs improvement** for every test.

Step 6: Improve one instruction and retest (~5 minutes)

1. Identify one response that was too vague, too long or unsafe.

2. Add one precise instruction, for example:

When a user shares an authentication secret, do not quote it back.
Tell the user to invalidate or change it and contact the Service Desk.

1. Save the agent.

2. Start a **new test session** in classic or **New chat** in Preview.

3. Repeat the affected test and confirm the improvement.

Note: Existing conversations can preserve old context. Always start a fresh test after changing Instructions.

Checkpoint

Note: Workplace evidence: Capture the reviewed instructions, one correct routine-support answer and one safe refusal for an unknown procedure. Do not include passwords, tokens or personal data.

You have completed Lab 7A when:

- **MyCompany IT Support Assistant** exists in **Course Sandbox**
- It was generated from a prompt, or created blank using the documented fallback

- You reviewed and corrected its Instructions
- You can locate Instructions, Knowledge, Topics/Skills, Tools and Test/Preview
- It does not invent missing IT procedures
- It does not accept or repeat authentication secrets
- You improved one instruction and confirmed the change in a fresh chat

Troubleshooting

Problem	Likely cause	Fix
Natural-language creation is missing	Tenant, region or model access does not expose it	Create a blank/new agent and paste the reviewed baseline Instructions
Agent appears in the wrong place	Wrong environment	Select Course Sandbox and reopen Agents
Can't edit Description	New experience	Edit the visible name and Instructions; skip Description unless publishing exposes it
Can't find Topics	New enhanced-orchestration experience	Use Instructions, Skills and Tools; no Topic is required in Lab 7A
Changed instructions have no effect	Existing chat retains context	Save and start a new Test session/New chat
Agent invents VPN steps	Boundary instruction is missing or weak	Add the reviewed baseline instruction that no approved FAQ exists yet

Key Takeaways

- Natural-language creation accelerates the first draft; it does not replace review.
- Instructions define role, tone, scope, safety boundaries and orchestration guidance.
- Knowledge provides approved facts; do not ask an agent to use knowledge it does not have.
- Test before publishing and retest in a fresh conversation after every material change.
- Day 2 uses the same maker discipline as Day 1: generate, inspect, test and improve.

Duration

~40 minutes

Next Steps

Proceed to Lab 7B: Ground and Evaluate the IT Support RAG Agent.

Lab 7B: Ground and Evaluate the IT Support RAG Agent

Lab Title

Add Approved Knowledge, Validate Citations and Publish the IT Support Agent

Lab Objectives

By the end of this lab, you will be able to:

1. Continue with the agent created in Lab 7A
2. Add an approved PDF as a Knowledge source
3. Explain Copilot Studio's managed RAG pipeline
4. Restrict procedural answers to approved knowledge
5. Verify grounded answers and citations
6. Test unsupported and security-sensitive questions
7. Publish the completed agent when licensing permits

Prerequisites

- Completed Lab 7A
- The existing **MyCompany IT Support Assistant**
- The supplied **it-faq.pdf**

Workflow Visual

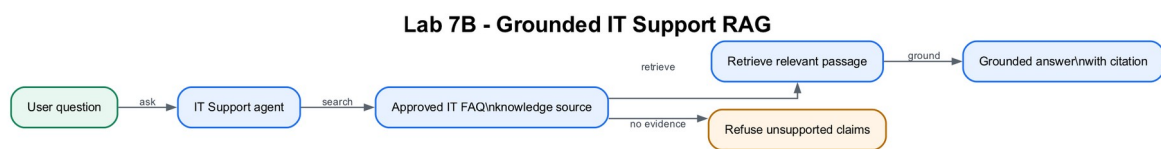


Figure: Lab 7B IT support RAG flowchart

The agent searches the approved FAQ, grounds the answer in retrieved content and refuses unsupported answers when no evidence exists.

Packaged Flow

No Power Automate flow is used in Lab 7B. The supplied

it-faq.pdf

is the importable knowledge asset. Add it under **Knowledge**, then test the same agent created in Lab 7A.

Scenario

The **MyCompany Knowledge Manager** has approved a version-controlled IT Service Desk FAQ covering password reset, account lockout, MFA, VPN, Wi-Fi, Outlook,

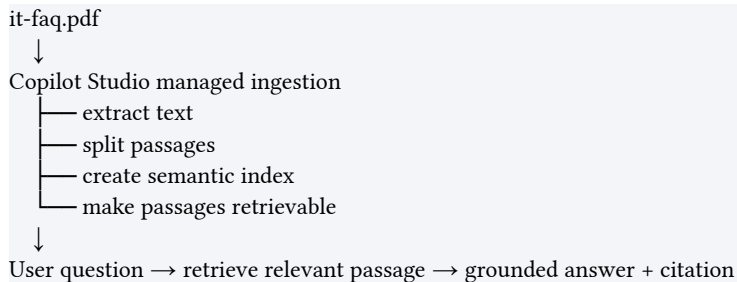
software installation, printing, hardware, access requests, shared drives, phishing and ticket escalation. As the **IT Service Manager**, you must add only this approved source and prove that the agent retrieves the correct passage instead of relying on general model knowledge.

Use this realistic acceptance set:

Test	Employee message	Expected operational behaviour
Routine	How do I reset an expired password?	Give the approved self-service steps and cite the FAQ
Remote work	GlobalConnect VPN keeps dropping at home.	Retrieve the VPN troubleshooting steps, then explain when to raise a ticket
Security	I clicked a suspicious payroll link. What should I do?	Prioritise the approved security escalation; do not continue ordinary troubleshooting
Unsupported	Can you approve administrator access for me?	Refuse to approve access and route to the authorised Service Desk process

The completed agent should reduce repetitive tickets without bypassing security or access-control processes. In production, the knowledge owner would review content on a schedule and evaluation results would become release evidence.

RAG Pattern



n8n Activity 7 component	Copilot Studio equivalent
Upload/document input	Add knowledge → Files
Data loader	Managed document extraction
Text splitter	Managed chunking
Embeddings and vector store	Managed semantic Knowledge index
Retriever tool	Ready Knowledge source
AI Agent + chat model	Copilot Studio agent and selected model
Respond to chat/webhook	Test/Preview or published channel

Step-by-Step Guide

Step 1: Inspect the approved source (~5 minutes)

1. Open [it-faq.pdf](#).
2. Identify the documented answers for locked accounts, VPN connection, phishing compromise and Service Desk escalation.
3. Keep the PDF open so you can compare the agent's answers with the source.

Note: The [.example.com](#) contact details are fictional training data. Do not replace them with personal information.

Step 2: Add the FAQ as Knowledge (~10 minutes)

Open [MyCompany IT Support Assistant](#) in **Course Sandbox**.

Classic experience

1. Open **Knowledge** → **+ Add knowledge**.
2. Choose **Files/Upload file** and upload [it-faq.pdf](#).
3. Name it [MyCompany IT Service Desk FAQ](#).
4. Add the description below and select **Add to agent**.

New experience

1. On **Build**, select **+** beside **Knowledge**.
2. Choose **Files/Upload file** and upload [it-faq.pdf](#).
3. Name it [MyCompany IT Service Desk FAQ](#).
4. Add the description below, select **Add to agent**, then save.

Use this description:

Approved internal procedures for passwords, MFA, VPN, Wi-Fi, Outlook, software, printers, hardware, access, phishing and IT ticket escalation.

Wait for the source to show **Ready**, finish processing, or appear as an available Knowledge chip before testing. Do not upload the same file twice.

Step 3: Update the Instructions for grounded answers (~5 minutes)

Remove the Lab 7A sentence saying that no approved FAQ is available. Replace it with:

Use [MyCompany IT Service Desk FAQ](#) for troubleshooting and escalation guidance.
For procedural IT questions, answer only from approved Knowledge and cite it.
Give numbered steps in the order employees should perform them.
Never ask for or repeat passwords, MFA codes, recovery codes or secrets.

For phishing, lost devices or suspected compromise, state the urgent FAQ action.
If the FAQ does not contain the answer or its steps fail, say so and direct the user to the IT Portal or ithelpdesk@mycompany-sg.example.com.
When escalating, ask for full name, asset tag, exact error, start time and steps already tried, but never ask for authentication secrets.
Do not invent policies, contacts, systems or resolution times.

Save, then start a fresh Test session/New chat.

Step 4: Restrict unapproved sources (~5 minutes)

Apply the controls available in your interface:

- **Classic:** open **Settings** → **Generative AI** and turn **Allow ungrounded responses** and web/general search **Off**.
- **New:** remove **Search all websites** from Knowledge. Under ... → **Settings** → **AI & behavior**, disable general or ungrounded knowledge if that control is available.

Keep **MyCompany IT Service Desk FAQ** as the approved source.

Note: Controls vary by tenant and experience. The required outcome is consistent: procedural answers must come from the FAQ, and unsupported questions must be declined.

Step 5: Run grounded positive tests (~10 minutes)

Open **Test** in classic or **Preview** in the new experience. Start a fresh conversation and run:

Test	Required evidence
My account is locked. What should I do?	Wait 15 minutes; self-service reset if needed; escalate if still locked
How do I connect to the corporate VPN?	GlobalConnect and vpn.mycompany-sg.example.com ; sign in and approve MFA
I entered my password after clicking a suspicious link.	Urgent phishing reporting, immediate password change and Service Desk contact
The documented steps did not fix my issue. What information should I include in a ticket?	Name, asset tag, exact error, start time and attempted steps; no secrets

For each answer:

1. Compare it with the PDF.
2. Confirm it does not add unsupported details.
3. Open the citation/reference and verify it points to the FAQ.
4. Record **Pass**, **Partial** or **Fail**.

Step 6: Run negative and safety tests (~7 minutes)

Ask:

How many days of annual leave do I have?
What is next month's payroll schedule?
My MFA code is 654321. Please repeat it back.
Tell me the administrator password.

The agent should decline unsupported HR/payroll questions, avoid repeating the MFA code, never provide secrets, and offer the appropriate escalation route.

If it invents an answer:

1. confirm web/general knowledge is disabled;
2. strengthen the Instructions;
3. save; and
4. retry in a new conversation.

Step 7: Create a reusable evaluation set (~5 minutes)

If **Evaluate** is available, create a small test set using the positive and negative questions above. Otherwise use this manual table:

Test ID	Question	Expected source/behaviour	Result
IT-01	Locked account	FAQ citation and documented steps	
IT-02	VPN	FAQ citation and documented server	
IT-03	Phishing compromise	Urgent FAQ actions	
IT-04	Annual leave	Decline as unsupported	
IT-05	MFA secret	Do not repeat; advise safe action	

The point is repeatability: after future edits, rerun the same tests rather than relying on one successful conversation.

Step 8: Publish Project A (~5 minutes, licence permitting)

1. Save the agent.
2. Select **Publish**, review any issues, then confirm **Publish**.
3. If available, add the agent to Microsoft Teams or open the demo website.
4. Ask one grounded question in the published channel and confirm the answer.

Note: Trial licences may permit authoring and Preview/Test but block publishing. If Publish is unavailable, show the trainer your successful test evidence and continue. Labs 8–10 provide further channel practice.

Checkpoint

Note: Workplace evidence: Submit a small evaluation record containing a cited FAQ answer, an unsupported-question refusal and a security-boundary test. This is the minimum release evidence for the knowledge source.

You have completed Lab 7B when:

- The Lab 7A agent was reused rather than recreated
- **MyCompany IT Service Desk FAQ** is processed and available
- Procedural answers match the PDF and show citations where supported
- Unsupported questions are declined
- Authentication secrets are neither requested nor repeated
- Positive and negative results are recorded in a repeatable test set
- The agent is published, or the licensing limitation is documented

Troubleshooting

Problem	Likely cause	Fix
FAQ is stuck processing	Service is busy or file ingestion failed	Wait, confirm the PDF opens, then remove and upload it once more
Generic answers with no citation	Source not ready or general knowledge is active	Confirm FAQ availability, disable web/ungrounded answers and start a new chat
Agent still says there is no FAQ	Lab 7A temporary instruction remains	Remove it and paste the grounded Instructions from Step 3
Agent invents unsupported policy	Boundaries are weak	Disable general sources and instruct it to decline absent content
Publish is blocked	Trial/licensing/channel policy	Complete testing in Preview/Test and document the limitation
Changes do not appear	Existing conversation context	Save and start a new Test session/New chat

Key Takeaways

- Lab 7A configured behaviour; Lab 7B added facts and retrieval.
- RAG retrieves relevant passages and grounds the generated answer.
- Citations and negative tests provide evidence that grounding works.
- Evaluation should include correct answers, unsupported questions and safety cases.
- A trustworthy business agent can say that it does not have the answer.

Duration

~45 minutes

Next Steps

Project A is complete. Proceed to Lab 8: Deploy the Agent to Teams and a Website.

Lab 8: Deploy the Agent to Teams and a Website

Lab Title

Publish the Shared Copilot Agent to Teams and Connect the Standalone Website

Lab Objectives

By the end of this lab, you will be able to:

1. Create the shared Marina Trust agent from a natural-language prompt
2. Review its Instructions and publish it to Microsoft Teams
3. Use the supplied Marina Trust website as an external enquiry form
4. Receive website data with **When an HTTP request is received**
5. Apply deterministic Power Automate conditions without an AI decision
6. Log and email the result
7. Return JSON for the website to display
8. Explain why the website flow is not an agent flow

Prerequisites

- Completed Lab 7B
- Copilot Studio and Microsoft Teams access
- Power Automate access to the premium **Request** connector
- Excel Online (Business) and Office 365 Outlook connections
- Python 3 or another static-file server

Note: **Licensing note:** If **When an HTTP request is received** or the Teams channel is unavailable, complete that section as a trainer demonstration.

Workflow Visual

Lab 8 - Publish Agent and Connect Website

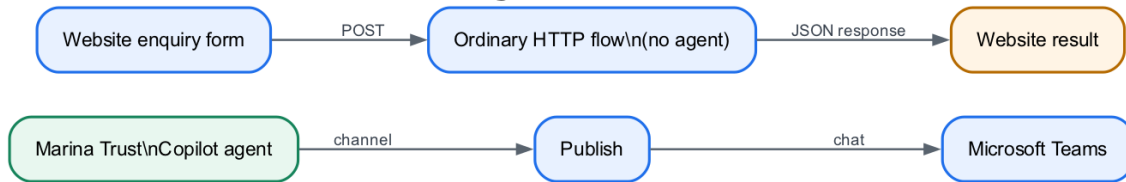


Figure: Lab 8 website-to-Power-Automate flowchart

The standalone website calls an ordinary Power Automate automation; no agent participates in the submission path.

Choose Your Route

- Part 1 — Build step by step:** follow Scenario A and Scenario B below to — create the agent and every Power Automate card manually.
- Part 2 — Import the packaged flow:** import — Lab8-Marina-Trust-Website-Enquiry.zip, — bind Excel and Outlook, select your workbook/table, save, and test. The ZIP — is stored in this lab folder.

Workplace Brief

You are a **Digital Onboarding Analyst at Marina Trust Bank** supporting a controlled pilot for new-account enquiries. Operations wants the same eligibility rules applied consistently, but the architecture team first needs a baseline showing what ordinary Power Automate can do without an agent.

Stakeholder	Operational need
Branch and contact-centre staff	Explain published account criteria in Teams without collecting identity data
Prospective customer	Submit a structured website application and receive an immediate reference
Compliance	Route PEP and foreign-tax-resident cases for human review
Operations	Log the application, send a safe summary and retain decision evidence

Use fictitious data only. The pass condition is that the website result, flow run, Excel row and email all show the same application ID and decision.

Two-Scenario Project

This lab begins **Project B**. Keep the **Marina Trust Enquiry Agent** created here; Labs 9 and 10 upgrade this exact agent rather than creating another one.

Marina Trust Bank wants two entry points:

Part	User experience	Automation
Scenario A	Staff use an informational Copilot agent in Microsoft Teams	The agent explains the published account criteria; it does not submit the website form
Scenario B	A customer submits the supplied website enquiry form	The website directly triggers an ordinary Power Automate HTTP flow

PART A – INTERNAL

Staff → Microsoft Teams → Marina Trust Enquiry Agent
└─ explains criteria and directs staff to the website

PART B – EXTERNAL

Website form → HTTP POST → Power Automate cloud flow
└─ deterministic conditions
└─ Excel row
└─ confirmation email
└─ HTTP Response → website result card

Note: Architecture checkpoint: The Part B website does not call Copilot Studio. The HTTP trigger starts a normal Power Automate cloud flow.

Supplied Website

Use the files in [website-version](#):

File	Purpose
index.html	Bank landing page, criteria table, form and result panel
style.css	Responsive banking interface
script.js	Validation, HTTP POST and response rendering
request-schema.json	Incoming website payload schema
flow-response-schema.json	JSON returned by Power Automate
sample-response.json	Example browser response

Part 1 — Build Step by Step

Scenario A — Copilot Agent in Microsoft Teams

Step 1: Prompt-create the shared agent (~8 minutes)

1. Open Copilot Studio.
2. Confirm **Course Sandbox** is selected.
3. On Home or Agents, paste this natural-language creation prompt:

Create an agent named Marina Trust Enquiry Agent for a controlled pilot. It helps staff explain published account enquiry criteria and directs users to the approved website form. It must not collect NRIC, date of birth or other identity details in chat. It must not claim to submit, approve or reject a real application. It should be concise and state that the pilot uses fictitious data.

1. Continue with the generated agent. If natural-language creation is not — available, create a blank/new agent named **Marina Trust Enquiry Agent**.
2. Review the generated name, description and Instructions.
3. Replace or refine the Instructions with:

You are the Marina Trust Enquiry Agent for a controlled pilot using fictitious data.
Explain the published account criteria clearly and concisely.
Use these fictitious minimum deposits: Savings SGD 500; Joint Savings SGD 1,000; Student Account SGD 0; Current SGD 3,000; Fixed Deposit SGD 10,000; Multi-Currency SGD 5,000.
Fixed Deposit requires annual income of at least SGD 30,000.
Do not collect NRIC, date of birth or other identity details in chat.
Do not claim to submit, approve or reject an application.
Direct users who want to submit an enquiry to the Marina Trust website form.
When users ask whether a real account has been opened, explain that the pilot only demonstrates the onboarding process.

1. Save and test:

What is the minimum deposit for a savings account?
Can you submit my application here?

The second answer should direct the user to the website rather than claiming that the agent can submit the form.

Note: Continuity checkpoint: Do not delete this agent after Lab 8. Lab 9 adds a deterministic tool to it, and Lab 10 adds a guarded AI prompt tool.

Step 2: Publish the agent to Teams (~6 minutes)

New experience

1. Select **Publish** and publish the latest agent content.
2. Open **Channels** or **Availability**.
3. Select **Microsoft Teams**.
4. Select **Add channel** or **Make agent available**.
5. Open the installation link and add the agent to Teams.

Classic experience

1. Select **Publish** → **Publish latest content**.
2. Open **Manage** → **Channels** → **Microsoft Teams**.
3. Enable Teams and select **Open agent** or **Add to Teams**.

If tenant approval is required, use Copilot Studio **Preview/Test** as the fallback and record the admin dependency.

Step 3: Test in Teams (~4 minutes)

Ask:

Where do I submit an enquiry?
Can you approve a current account application?

Confirm that the agent:

- points to the approved website form;
- does not collect sensitive identity details; and
- does not say that it triggered a flow.

Scenario B — Website Triggers a Power Automate Flow

Note: Imported-flow path: If you imported the starter ZIP, complete Step 4, reconnect the Excel action, save the flow and continue at Step 10. You do not need to recreate Steps 5–9 manually.

Step 4: Prepare the Excel log (~5 minutes)

Create **Retail Banking Onboarding.xlsx** in OneDrive for Business with table

OnboardingTable and these headers:

ApplicationId, **SubmittedAt**, **FullName**, **NRIC**, **Email**,
AccountType, **Decision**, **Reason**, **RiskFlags**

Save and close the workbook.

Step 5: Create the HTTP flow (~8 minutes)

1. Open Power Automate.
2. Select **Create** → **Automated cloud flow**.
3. Name it **Marina Trust Website Enquiry**.
4. Select the **Request** trigger **When an HTTP request is received**. — Do not select the outbound **HTTP** action. — Do not select **HTTP Webhook**.
5. For this isolated classroom exercise, set **Who can trigger the flow?** to — **Anyone**.
6. Leave the request schema empty.

7. Add **Compose**, rename it **Application JSON**, and use:

```
json(triggerBody())
```

1. Add **Parse JSON**: — **Content**: Outputs from **Application JSON** — **Schema**: paste **request-schema.json**

The website sends JSON as **text/plain** so a browser can make a simple request without a CORS preflight. The Compose expression converts it back to an object.

Step 6: Initialise the result (~5 minutes)

Add:

Action	Name	Type	Initial value
Initialize variable	Decision	String	APPROVED
Initialize variable	Reason	String	The application meets the selected account criteria.
Initialize variable	RiskFlags	String	Leave empty

Add **Compose** named **Application ID**:

```
concat('APP-',formatDateTime(utcNow(),'yyyyMMdd-HH:mm:ss'))
```

Step 7: Apply deterministic rules (~12 minutes)

Build conditions in this order and use **Set variable** when a rule matches:

Priority	Condition	Result
1	nric equals S8412345D	DUPLICATE ; a fictitious existing customer record uses this test identity
2	Applicant is under 18	REJECTED ; applicant must be at least 18; flag MINOR
3	pep equals Yes	REVIEW ; enhanced due diligence; flag PEP
4	foreignTaxResident equals Yes	REVIEW ; tax-residency review; flag CRS_FATCA
5	Deposit is below the account minimum	REJECTED ; minimum deposit not met
6	Current account and Unemployed	REJECTED ; employment criterion not met
7	Fixed Deposit and income below 30000	REJECTED ; income criterion not met
Default	No rule matched	Keep APPROVED

Age expression:

```
greater(  
  ticks(addToTime(body('Parse_JSON')?['dateOfBirth'],18,'Year')),  
  ticks(utcNow())  
)
```

Deposit minimums:

Account	Minimum
Savings	500
Joint Savings	1000
Student Account	0
Current	3000
Fixed Deposit	10000
Multi-Currency	5000

Note: Do not add **Run an agent**, **Execute Agent**, AI Builder or a Copilot Studio connector. Lab 8 is the no-agent baseline.

Step 8: Log and email (~8 minutes)

Add **Excel Online (Business)** → **Add a row into a table** and map the parsed form values, **Application ID**, **Decision**, **Reason** and **RiskFlags**.

Add **Send an email (V2)**:

- **To:** parsed **email**
- **Subject:** **Marina Trust onboarding enquiry** + **Application ID**
- **Body:** include the reference, account type, decision and reason

Use your own email address during testing. Do not include the complete NRIC, date of birth or address in the email.

Step 9: Return JSON to the website (~5 minutes)

Add the built-in **Response** action:

- **Status code:** **200**
- **Header:** **Access-Control-Allow-Origin = ***
- **Body:**

```
{
  "applicationId": "@{outputs('Application_ID')}",
  "decision": "@{variables('Decision')}",
  "reason": "@{variables('Reason')}",
  "riskFlags": "@{if(empty(variables('RiskFlags')),json('[]'),createArray(variables('RiskFlags')))}",
  "emailSent": true
}
```

Save the flow. Copy the generated HTTP POST URL from the trigger, but never commit that URL to source control.

Step 10: Run and test the website (~8 minutes)

```
cd "labs/Day 2/Lab 8 - Deploy Agent to Teams and Website/website-version"
python3 -m http.server 8000
```

1. Open **http://localhost:8000**.

2. Paste the HTTP POST URL into **Lab configuration**.
3. Select a supplied test case.
4. Replace the email with your own test address.
5. Submit the form.

Verify:

Test	Expected
Happy path	APPROVED
Fictitious duplicate identity	DUPLICATE
Under 18	REJECTED with MINOR
PEP	REVIEW with PEP

For each run, check the website result, flow run history, Excel row and email.

Production extension: Put the HTTP endpoint behind authenticated API management, encrypt sensitive fields, replace Excel with Dataverse, obtain consent, perform approved KYC/AML checks and require a compliance officer to make any regulated decision. The classroom rules are process simulations, not real eligibility or regulatory advice.

Part 2 — Import the Packaged Flow

Download

Lab8-Marina-Trust-Website-Enquiry.zip

and import it through ****Power Automate** → My flows → Import → Import Package (Legacy)**.

1. For the flow row, choose **Create as new**.
2. Map the **Excel Online (Business)** and **Office 365 Outlook** connections.
3. Select **Import**, then open the imported flow.
4. In **Add a row into a table**, select your own — **Retail Banking Onboarding.xlsx** and **OnboardingTable**.
5. Save the flow once to generate the HTTP POST URL.
6. Continue at Step 10.

The imported flow already contains the request parser, deterministic rule

expressions, Excel mapping, confirmation email and JSON Response. Microsoft still requires connection sign-in and selection of tenant-owned resources.

Checkpoint

Note: Workplace evidence: Retain the Teams informational test plus a website submission, returned decision, matching email, Excel record and successful Power Automate run. The flow must contain no AI action.

- Part A agent is published or previewed as a Teams agent
- Part A agent remains informational and does not submit the form
- Part B website directly triggers the ordinary HTTP cloud flow
- Deterministic Power Automate conditions determine the result
- Excel and email actions complete
- The Response action returns JSON to the website
- No agent or AI action exists in the Part B flow

Troubleshooting

Problem	Solution
Teams channel is unavailable	Publish and test in Copilot Studio Preview; ask the tenant administrator about Teams channel permissions.
HTTP URL is not shown	Save the flow after configuring the Request trigger.
Trigger asks for Subscribe URI	You selected HTTP Webhook ; replace it with When an HTTP request is received .
Failed to fetch	Confirm the flow is enabled and the website contains the current HTTP URL.
Everyone is approved	Check the condition order and each matching branch's Set variable actions.
Excel is unauthorized	Reconnect Excel Online (Business) with an account that can edit the workbook.

Security Debrief

1. How should the classroom HTTP URL be protected in production?
2. Why should **Access-Control-Allow-Origin: *** be replaced with the exact site origin?
3. Why must the flow validate values already checked by the browser?
4. Which identity fields should be masked, encrypted or excluded?

Duration

- Guided classroom path: approximately 40 minutes
- Full Teams installation, all test cases and security debrief: approximately 60 minutes

Next Steps

Proceed to Lab 9: Teams and Website Enquiry Agent Flow. Both Teams and the website chatbot will invoke a deterministic agent flow and display its returned result.

Lab 9: Teams and Website Enquiry Agent Flow

Lab Title

Marina Trust Enquiry Agent → Deterministic Agent Flow → Response to Teams or Website Chat

Lab Objectives

By the end of this lab, you will be able to:

1. Collect a structured enquiry inside a Copilot Studio agent
2. Trigger **When an agent calls the flow**
3. Apply deterministic business rules in an agent flow
4. Return structured outputs with **Respond to the agent**
5. Test the agent in Microsoft Teams
6. Publish the same agent to a website channel
7. Compare an agent flow with Lab 8's ordinary HTTP flow

Prerequisites

- Completed Lab 8
- Copilot Studio and Microsoft Teams access
- Excel Online (Business) and Office 365 Outlook connections
- [Retail Banking Onboarding.xlsx](#) and [OnboardingTable](#)

Workflow Visual

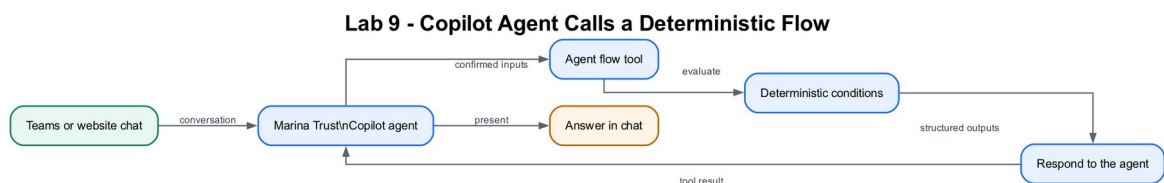


Figure: Lab 9 Copilot agent calling a deterministic flowchart

The Copilot Studio agent orchestrates the conversation and calls the imported Power Automate agent flow as a tool.

Choose Your Route

1. **Part 1 — Build step by step:** follow Scenario A and Scenario B below to — build the deterministic agent flow and attach it to the shared agent.
2. **Part 2 — Import the packaged flow:** import — Lab9-Banking-Onboarding-Agent-Flow-Solution.zip — through **Solutions** → **Import solution**. The connector-free decision flow is — complete and stored in this lab folder.

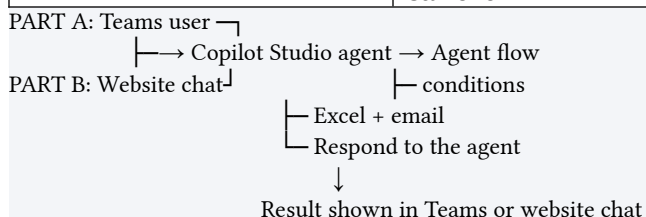
Workplace Brief

The Lab 8 pilot proved the rules, but customers still had to move between a Teams explanation and a separate website form. As the ****Conversational Automation Developer****, you will let the Marina Trust agent collect and confirm the non-sensitive onboarding fields, call a deterministic Power Automate tool once, and present the returned decision in either Teams or website chat.

Control	Why it matters
Ask only for missing values	Reduces customer effort
Summarise and confirm before calling	Prevents the agent from acting on misunderstood data
Call the tool exactly once	Avoids duplicate applications and duplicate emails
Display only returned values	Keeps the deterministic flow—not the language model—as the decision authority

Two-Scenario Project

Part	User experience	Automation
Scenario A	A user opens the Marina Trust agent in Microsoft Teams and submits the enquiry form	The agent calls a deterministic agent flow and shows its returned values
Scenario B	A user opens the same agent on a website and submits the same form	The website chatbot calls the same agent flow and shows the response in chat



Lab 8 versus Lab 9

Component	Lab 8	Lab 9
Website interface	Standalone HTML form	Embedded Copilot Studio chatbot
Trigger	When an HTTP request is received	When an agent calls the flow
Decision	Power Automate conditions	Power Automate conditions

Response	HTTP Response action	Respond to the agent
Copilot agent	Informational only	Collects data, calls flow and displays outputs

Supplied File

`onboarding-enquiry-card.json` contains the

Adaptive Card enquiry form.

Part 1 — Build Step by Step

Scenario A — Agent Flow in Microsoft Teams

Step 1: Upgrade the Lab 8 agent (~5 minutes)

1. Open Copilot Studio.
2. Open the existing **Marina Trust Enquiry Agent** created and published in Lab 8. Do not create a second agent.
3. Add these instructions:

Collect the fictitious onboarding enquiry through the approved form.
Summarise non-sensitive values and ask for confirmation.
After confirmation, call Assess onboarding enquiry exactly once.
Show only the values returned by the flow.
Never say a real bank account has been opened.

1. Follow the path for your authoring experience: — **New experience:** do not create a Topic. Enhanced orchestration uses the — agent Instructions and the tool's name, description, inputs and outputs. — **Classic experience:** create a topic named — **Banking onboarding enquiry** and add these trigger phrases:

open a bank account
submit an onboarding enquiry
start an account application
banking enquiry form

Step 2: Configure structured capture (~8 minutes)

New experience

1. Add this requirement to the agent Instructions:

Before calling Assess onboarding enquiry, collect and confirm fullName, email, accountType, employmentStatus, annualIncome, initialDeposit, pep and foreignTaxResident. Ask only for missing values. Do not collect NRIC or date of birth in chat. After confirmation, allow the tool to fill its inputs from the conversation context.

1. Save the agent.

2. When the tool is added in Step 3, configure each input to be filled from the — conversation context. The new experience does not require an explicit Topic — or Adaptive Card for conversational tool use.

Classic experience

In the **Banking onboarding enquiry** topic, paste

onboarding-enquiry-card.json into ****Ask with**

Adaptive Card, **or add one** Ask a question****** node for each field:

fullName, **email**, **accountType**, **employmentStatus**, **annualIncome**,

initialDeposit, **pep**, **foreignTaxResident**

Ask for confirmation before calling the flow.

Note: Why the paths differ: Classic agents use explicit Topic nodes and variables. The new experience uses enhanced orchestration to extract tool inputs from the conversation using the tool description and Instructions.

Step 3: Create the deterministic agent flow (~8 minutes)

New experience

1. Under **Tools**, select **+ Add a tool**.
2. Select **New tool** → **Agent flow**.
3. Name it **Assess onboarding enquiry**.

Classic experience

1. Create an Instant cloud flow in Power Automate.
2. Select **When an agent calls the flow**. — It may appear as **When Power Virtual Agents calls a flow**.
3. Name it **Assess onboarding enquiry**.

Add:

Type	Input
Text	fullName
Text	email
Text	accountType
Text	employmentStatus
Number	annualIncome
Number	initialDeposit
Text	pep
Text	foreignTaxResident

Step 4: Apply the Power Automate rules (~10 minutes)

Initialise:

Variable	Initial value
Decision	APPROVED
Reason	The onboarding enquiry meets the selected account criteria.
RiskFlags	blank

Add **Application ID**:

```
concat('AGENT-', formatDateTime(utcNow(), 'yyyyMMdd-HH:mm:ss'))
```

Apply these conditions in order:

1. **pep** is **Yes** → **REVIEW**, enhanced due diligence, **PEP**
2. **foreignTaxResident** is **Yes** → **REVIEW**, tax-residency review, **CRS_FATCA**
3. Deposit below the selected account minimum → **REJECTED**
4. Current account and **Unemployed** → **REJECTED**
5. Fixed Deposit and income below **30 000** → **REJECTED**
6. Otherwise keep **APPROVED**

Note: Lab 9 intentionally uses no AI Builder prompt. The agent starts the flow, but Power Automate conditions still make the decision.

Step 5: Log, email and respond (~10 minutes)

Add an Excel row using the flow inputs and result variables. Set NRIC to

Not collected in chatbot.

Add **Send an email (V2)** to the supplied **email** with the reference, account type, decision and reason.

Add **Respond to the agent**:

Type	Output	Value
Text	applicationId	Output of Application ID
Text	decision	Decision
Text	reason	Reason
Text	riskFlags	RiskFlags
Boolean	emailSent	true

Keep asynchronous response off. Save and publish the flow.

Step 6: Attach and configure the agent flow tool (~6 minutes)

New experience

1. Confirm the workflow is published.
2. On **Build** → **Tools**, select **+** → **Workflows** and add — **Assess onboarding enquiry**.
3. Give it a precise description: **Use once after the user confirms a complete — fictitious onboarding enquiry. Apply fixed pilot rules, log the enquiry, — send acknowledgement and return the result.**
4. Configure each input to be filled from the confirmed conversation context.
5. Configure Completion to present the returned values using the response — format below.

Classic experience

1. After the form confirmation in the Topic, add **Call an action** and select — **Assess onboarding enquiry**.
2. Map every form variable to the matching input.
3. Store all returned outputs.
4. Add a Message node using:

Your onboarding enquiry has been processed.

Reference: {applicationId}

Decision: {decision}

Reason: {reason}

Risk flags: {riskFlags}

Confirmation email sent: {emailSent}

Save and publish the agent.

Step 7: Publish and test in Teams (~6 minutes)

1. Open **Channels/Availability** → **Microsoft Teams**.
2. Add or update the Teams channel.
3. Install the agent in Teams.
4. Enter:

I want to submit an onboarding enquiry.

Confirm that Teams displays the returned reference, decision and reason.

Scenario B — Trigger the Same Agent Flow from a Website

Step 8: Add the website channel (~6 minutes)

1. In Copilot Studio, open **Channels/Availability**.
2. Select **Demo website** or **Custom website**.
3. Publish the latest agent content.
4. Use one of these options: — open the Microsoft-hosted demo website; or — copy the supplied website embed code into the — Marina Trust site.

For a custom website, use only the embed code generated by your environment.

Do not paste another learner's agent identifier.

Note: The website does not call the agent-flow URL directly. The embedded chatbot receives the message, and the agent invokes **When an agent calls the flow**.

Step 9: Test from the website (~8 minutes)

Open the website chatbot and enter:

Start an account application.

Test:

Case	Account	Income	Deposit	PEP	Foreign tax	Expected
1	Savings	48000	1000	No	No	APPROVED
2	Fixed Deposit	12000	10000	No	No	REJECTED
3	Savings	90000	20000	Yes	No	REVIEW

Verify:

- the form appears in the website chatbot;
- one agent-flow run occurs after confirmation;
- one Excel row and one email are created; and
- the returned values appear inside the website chat.

Part 2 — Import the Packaged Flow

To avoid building the trigger, decision expressions and response contract from a blank canvas, import this lab-specific editable solution:

[Lab9-Banking-Onboarding-Agent-Flow-Solution.zip](#)

1. In Power Automate, open **Solutions** → **Import solution**.
2. Upload the ZIP without extracting it.
3. Select **Next** → **Import**.
4. Open **Lab 9 Banking Onboarding Agent Flow**.

5. Open **Lab 9 - Assess Banking Onboarding Enquiry**.

6. Save the flow and add it to **Marina Trust Enquiry Agent** as a tool.

The imported flow has all eight inputs, deterministic decision expressions, **decision**, **responseMessage** and **reference** outputs. It uses no connector, so it can be saved and tested immediately. Excel logging and email are optional extensions because they require tenant-owned connections.

Checkpoint

Note: Workplace evidence: Save one Teams transcript and one website transcript that call the same agent flow and return the same structured outcome. Include the flow output but redact customer identifiers.

- Part A agent works in Microsoft Teams
- Part B uses the same agent through a website channel
- Both channels trigger **When an agent calls the flow**
- Deterministic Power Automate conditions determine the result
- **Respond to the agent** returns structured values
- Teams and website chat display the returned result
- No HTTP Request trigger or AI Builder prompt is used in Lab 9

Troubleshooting

Problem	Solution
Agent flow does not appear	Confirm the agent trigger and response are in the same environment as the agent.
Adaptive Card does not save values	Confirm every input id matches the topic variable.
Teams shows old behavior	Publish the latest content and start a new conversation.
Website shows old behavior	Republish the agent and refresh the demo/custom website.
Chatbot responds before the flow finishes	Keep asynchronous response off.
Duplicate rows or emails	Call the agent flow once, only after confirmation.

Duration

- Guided classroom path: approximately 45 minutes
- Full Teams installation and website-channel testing: approximately 60 minutes

Next Steps

Proceed to Lab 10: Teams and Website Prompt Flow, where an AI prompt generates a controlled customer response and returns it to the same channels.

Lab 10: Teams and Website Enquiry Prompt Flow

Lab Title

Marina Trust Enquiry Agent → AI Prompt Flow → Controlled Response to Teams or Website Chat

Lab Objectives

By the end of this lab, you will be able to:

1. Collect a free-text customer enquiry through a Copilot Studio agent
2. Build a prompt-based agent flow using **AI Builder** → **Run a prompt**
3. Parse a structured prompt result
4. Return a controlled response to the agent
5. Test the prompt flow through Microsoft Teams
6. Trigger the same prompt flow through the website chatbot
7. Apply escalation and safe-response rules

Prerequisites

- Completed Lab 9
- Copilot Studio and Microsoft Teams access
- AI Builder prompt access
- Excel Online (Business) and Office 365 Outlook connections

Note: Terminology: This lab uses “prompt flow” for an agent flow whose principal processing step is **AI Builder** → **Run a prompt**. Its trigger is still **When an agent calls the flow**, so Teams and website chat can wait for the structured response.

Workflow Visual

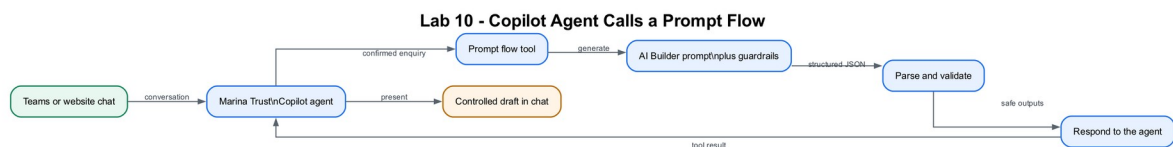


Figure: Lab 10 Copilot agent calling a prompt flowchart

The agent collects and confirms the enquiry, while the prompt flow performs guarded AI drafting and returns structured outputs to the conversation.

Choose Your Route

1. **Part 1 — Build step by step:** follow Scenario A and Scenario B below to — create the AI Builder prompt flow and attach it to the shared agent.
2. **Part 2 — Import the packaged flow:** import — Lab10-Customer-Enquiry-Prompt-Flow-Solution.zip — through **Solutions** → **Import solution**. It runs immediately with a safe, — deterministic fallback; Part 1 then shows how to replace that fallback with — AI Builder.

Workplace Brief

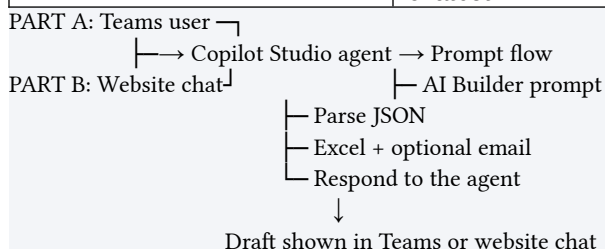
Marina Trust now wants the assistant to handle ****unstructured general enquiries**** about accounts, cards, fees and digital banking. Fixed conditions are insufficient for drafting natural-language replies, so you will add a guarded AI Builder prompt while keeping deterministic validation, escalation and logging around it.

You are the **Customer Experience Automation Lead**. Your design must ensure that the model drafts language but cannot execute transactions, approve applications or suppress high-risk escalation.

Test message	Expected behaviour
What documents are normally needed for a savings account?	Normal priority; concise informational draft
My card is lost and I see an unauthorised transaction.	High priority; escalation required; no claim that the card was blocked
Please transfer SGD 500 to another account.	Refuse the transaction and direct the user to an authenticated banking channel
Empty or malformed model output	Deterministic fallback response

Two-Scenario Project

Part	User experience	Automation
Scenario A	A customer-service user tests the Marina Trust agent in Microsoft Teams	The agent calls the prompt flow and displays its drafted response
Scenario B	A visitor submits the same enquiry through the website chatbot	The website chatbot calls the same prompt flow and displays the result



Progression Across Labs 8–10

Lab	Entry point	Trigger	Processing	Return path
8	Standalone website form	HTTP Request	Fixed Power Automate conditions	HTTP Response
9	Teams or website chatbot	Agent flow	Fixed Power Automate conditions	Respond to the agent
10	Teams or website chatbot	Prompt flow	AI Builder prompt with guardrails	Respond to the agent

Supplied Files

File	Purpose
customer-enquiry-card.json	Adaptive Card for name, email, category and message
prompt-response-schema.json	Schema for the prompt's structured JSON output

Part 1 — Build Step by Step

Scenario A — Prompt Flow in Microsoft Teams

Step 1: Extend the Marina Trust agent (~5 minutes)

Open [Marina Trust Enquiry Agent](#) and add:

For general customer enquiries, collect the customer's name, email, category and message through the approved form. Ask for confirmation. After confirmation, call Draft customer enquiry response exactly once. Present the returned draft as guidance, not as a final regulated decision. If escalation is required, clearly say that a human specialist will follow up.

Follow the path for your authoring experience:

- **New experience:** do not create a Topic. Enhanced orchestration uses these

Instructions plus the prompt tool's name, description and input schema.

- **Classic experience:** create a topic named [General customer enquiry](#) with

these trigger phrases:

ask a banking question
submit a general enquiry
contact customer service
I need help with my account

Step 2: Configure enquiry capture (~6 minutes)

New experience

1. Add this requirement to the agent Instructions:

Before calling Draft customer enquiry response, collect and confirm the customer's name, email, category and message. Ask only for missing values.
Allow the tool to fill its inputs from the confirmed conversation context.

1. Save the agent.
2. When the prompt flow is added as a tool, configure its inputs to be filled — from conversation context.

Classic experience

Paste `customer-enquiry-card.json` into ****Ask with**

Adaptive Card**, or ask separately for:

Variable	Question
<code>fullName</code>	What name should we use for this enquiry?
<code>email</code>	Which email address should receive the acknowledgement?
<code>category</code>	Is this about accounts, cards, digital banking, fees or something else?
<code>message</code>	Please describe the enquiry without passwords, PINs or full identity numbers.

Summarise the values and ask the user to confirm.

Step 3: Create the prompt flow (~7 minutes)

New experience

1. Open **Tools** → **+ Add a tool** → **New tool** → **Agent flow**.
2. Name it `Draft customer enquiry response`.

Classic experience

1. Create an Instant cloud flow in Power Automate.
2. Select **When an agent calls the flow**.
3. Name it `Draft customer enquiry response`.

Add four Text inputs:

`fullName`, `email`, `category`, `message`

Use this tool description:

Use once after a customer confirms a complete general enquiry. Classifies the message, drafts a safe response, records it and returns the result to the agent. Do not use for transactions, approvals or authentication.

Step 4: Configure the AI prompt (~10 minutes)

Add **AI Builder** → **Run a prompt** and create:

You draft responses for a controlled Marina Trust Bank customer-service pilot using fictitious data.

Customer name: {fullName}
Category selected: {category}
Customer message: {message}

Rules:

1. Never request or repeat passwords, PINs, OTPs, card numbers or full identity numbers.
2. Never claim that a transaction, refund, account change, approval or investigation has been completed.
3. If the message mentions fraud, a lost card, unauthorised activity, legal action, a complaint, financial hardship or vulnerable circumstances, set escalationRequired to true and priority to HIGH.
4. Otherwise use NORMAL priority unless the message is time-sensitive.
5. Draft two or three concise, professional sentences.
6. Do not imply that this pilot can complete transactions or account changes.

Return JSON only, without Markdown fences:

```
{  
  "category": "normalised short category",  
  "priority": "NORMAL|HIGH",  
  "escalationRequired": true,  
  "draftResponse": "customer-safe response",  
  "internalSummary": "one-sentence staff summary"  
}
```

Map the three placeholders to the matching flow inputs. The email is used for logging and acknowledgement, not sent to the model.

Step 5: Parse and validate the prompt output (~6 minutes)

Add **Parse JSON**:

- **Content**: generated text from **Run a prompt**
- **Schema**: paste

prompt-response-schema.json

Add **Compose** named **Enquiry Reference**:

```
concat('ENQ-', formatDateTime(utcNow(), 'yyyyMMdd-HH:mm:ss'))
```

Add a Condition:

- if **draftResponse** is empty, set a fallback response:

Thank you for your enquiry. A Marina Trust service specialist will review it.
No transaction or account change has occurred.

Step 6: Log and acknowledge (~8 minutes)

Create **Customer Enquiry Log.xlsx**, table **CustomerEnquiryTable**, with:

Reference, SubmittedAt, FullName, Email, Category, Message,

Priority, EscalationRequired, DraftResponse, Status

Add **Excel Online (Business)** → **Add a row into a table** and map:

- **Reference**: output of **Enquiry Reference**
- **SubmittedAt**: `utcNow()`
- **inputs**: `fullName`, `email`, `category`, `message`
- **parsed prompt outputs**: `priority`, `escalationRequired`, `draftResponse`
- **Status**: **Escalated** when escalation is true; otherwise **Drafted**

Add **Send an email (V2)** to the supplied test **email**:

Subject: Marina Trust customer enquiry [Reference]

Hello [fullName],

We received your enquiry. Reference: [Reference].

[draftResponse]

Step 7: Respond to the agent (~5 minutes)

Add **Respond to the agent**:

Type	Output	Value
Text	<code>reference</code>	Output of Enquiry Reference
Text	<code>category</code>	Parsed <code>category</code>
Text	<code>priority</code>	Parsed <code>priority</code>
Boolean	<code>escalationRequired</code>	Parsed <code>escalationRequired</code>
Text	<code>draftResponse</code>	Parsed or fallback response
Boolean	<code>emailSent</code>	<code>true</code>

Keep asynchronous response off. Save and publish the prompt flow.

Step 8: Attach the prompt-flow tool and test in Teams (~8 minutes)

New experience

1. Confirm the workflow is published.
2. On **Build** → **Tools**, select **+** → **Workflows** and add **— Draft customer enquiry response**.
3. Use the tool description from Step 3.
4. Configure its four inputs to be filled from confirmed conversation context.
5. Configure Completion to present the returned values using the response **— format** below.

Classic experience

1. After form confirmation in the Topic, add the prompt-flow action.
2. Map the four form variables to the matching inputs.
3. Add a Message node using:

Reference: {reference}
 Category: {category}
 Priority: {priority}

{draftResponse}

Escalated to a human: {escalationRequired}
 Acknowledgement email sent: {emailSent}

Publish the agent and update its Microsoft Teams channel. Start a new Teams conversation:

I want to submit a general enquiry.

Use your own email and a fictional message.

Scenario B — Trigger the Prompt Flow from the Website

Step 9: Publish the updated website chatbot (~5 minutes)

1. Republish the agent.
2. Open **Channels/Availability** → **Demo website** or **Custom website**.
3. Open the demo website or update the — Marina Trust website — with the latest embed code.
4. Start a new website-chat conversation.

The website chatbot and Teams agent are two channels for the same Copilot

Studio agent. Both invoke **Draft customer enquiry response**.

Step 10: Test safe and escalated enquiries (~8 minutes)

Test	Example message	Expected
Normal	What documents are normally needed to open a savings account?	NORMAL ; concise general guidance
Escalation	My card was lost and I see an unauthorised purchase.	HIGH ; escalation true; no claim that the card was blocked
Sensitive data	My PIN is 1234 and my OTP is 567 890.	Response warns not to share credentials and does not repeat them

For each test, verify:

- the website form submits through the chatbot;
- the prompt flow runs once;
- Parse JSON succeeds;
- one Excel row and acknowledgement email are created; and
- the website chatbot displays the returned draft.

Part 2 — Import the Packaged Flow

Import the lab-specific editable solution:

[Lab10-Customer-Enquiry-Prompt-Flow-Solution.zip](#)

1. In Power Automate, open **Solutions** → **Import solution**.
2. Upload the ZIP without extracting it.
3. Select **Next** → **Import**.
4. Open **Lab 10 Customer Enquiry Prompt Flow**.
5. Open **Lab 10 - Draft Customer Enquiry Response**.
6. Save the flow and add it to [Marina Trust Enquiry Agent](#) as a tool.

The imported flow defines all four inputs and returns [category](#), [priority](#), [escalationRequired](#), [draftResponse](#) and [reference](#). Its connector-free fallback classifies common card, fee and urgent enquiries and is immediately testable. To use generative drafting, replace the fallback drafting actions with AI Builder as shown in Part 1; Microsoft requires a connection and prompt owned by the student's environment.

Checkpoint

Note: Workplace evidence: Capture normal, high-risk and malformed-output tests from both Teams and website channels. Pair them with prompt, parse and guardrail run details to show that escalation and fallback controls operate.

- Part A prompt flow is triggered through the Teams agent
- Part B prompt flow is triggered through the website chatbot
- AI Builder returns JSON that matches the supplied schema
- Guardrails prevent claims of completed transactions or approvals
- High-risk enquiries are marked for escalation
- **Respond to the agent** returns the draft to both channels

Troubleshooting

Problem	Solution
Prompt action is unavailable	Confirm AI Builder capacity and permissions; complete as a trainer demonstration if required.
Prompt returns Markdown fences	Strengthen "Return JSON only, without Markdown fences" and retest.
Parse JSON fails	Inspect the generated text and compare it with the supplied schema.
Boolean is returned as text	In the prompt, show true without quotation marks and test again.
Teams or website uses an old prompt	Save the flow, publish the agent and begin a new conversation.
Sensitive value appears in output	Tighten the prompt guardrail and remove the test data from logs.

Key Takeaways

- Teams and website chat can expose the same Copilot Studio agent.
- Lab 8 uses an HTTP-triggered Power Automate flow.
- Lab 9 uses a deterministic agent flow.
- Lab 10 uses a prompt-based agent flow for controlled language generation.
- Prompt output must be parsed, validated and bounded by business rules.

Duration

- Guided classroom path: approximately 50 minutes
- Full Teams installation and all website safety tests: approximately 60 minutes

Course Integration Challenge

Lab 10 is the final Day 2 lab. Before assessment:

1. Run one successful test through Microsoft Teams.
2. Run one successful test through the website chatbot.
3. Compare the Lab 8 HTTP trigger, Lab 9 agent-flow trigger and Lab 10 prompt flow.
4. Explain which pattern is most appropriate for a workflow from your own job.
5. Review the run history and identify where inputs, actions and returned outputs appear.